

COPPERBELT UNIVERSITY
COMPUTER SCIENCE DEPARTMENT
COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)
LECTURE NOTES

UNIT 1: PRELIMINARIES

INTRODUCTION TO THE THEORY OF COMPUTATION

Why study theory

The first answer is that theory provides concepts and principles that help us understand the general nature of the discipline. The field of computer science includes a wide range of special topics, from machine design to programming. The use of computer in the real world involves a wealth of specific detail that must be learned for a successful application. This makes computer science a very diverse and broad discipline. But in spite of this diversity, there are some common underlying principles. To study these basic principles, we construct abstract models of computers and computation. These models embody the important features that are common to both hardware and software, and that are essential to many of the special and complex constructs we encounter while working with computers. Even when such models are too simple to be applicable immediately to real-world situations, the insights we gain from studying them provide the foundations on which specific development is based. This approach is of course not unique to computer science. The construction of models is one of the essentials of any scientific discipline, and the usefulness of a discipline is often dependent on the existence of simple, yet powerful, theories and laws,

A second, and perhaps not so obvious answer, is that the ideas we will discuss have some immediate and important applications. The fields of digital design, programming languages, and compilers are the most obvious examples, but there are many others. The concepts we study here run like a thread through much of computer science from operating systems to pattern recognition.

In this course, we will look at models that represent features at the core of all computers and their applications. To model the hardware of a computer, we introduce the notion of an automaton (plural, automata). An automaton is a construction that possesses all the indispensable features of a digital computer. It accepts input, produces output, may have some temporary storage, and make decisions in transforming the input into output. A formal language is an abstraction of the general characteristics of programming languages. A formal language consists of a set of symbols and some rules of formation by which these symbols can be combined into

entities called sentences. A formal language is the set of all strings permitted by the rules of formation. Although some of the formal languages we study here are simpler than programming languages, they have many of the same essential features. Finally we will formalize the concept of a mechanical computation by a precise definition of the term **algorithm** and study the kinds of problems that are (and are not) suitable for solution by such mechanical means. In the course of our study, we will show the close connection between these abstractions and investigate the conclusions we can derive from them.

MATHEMATICAL PRELIMINARIES AND NOTATION

SETS

A set is a well-defined collection of objects, e.g. the set of all students in a college. The collection of all books in a library is also a set. (The individual objects are called members or elements of the set.)

We use capital letters A, B, C, ... for denoting sets. The small letters a, b, c... are used to denote elements of any set. When a is an element of the set A, we write $a \in A$. The statement that a is not in A is written $a \notin A$.

Ways of Describing a Set

- [i] *By listing its elements.* We list all the elements of the set (without repetition) and enclosing them within braces ($\{\}$). We can write the elements in any order. For example, the set of all positive integers divisible by 15 and less than 100 can be written as $\{15, 30, 45, 60, 75, 90\}$
- [ii] *By describing the properties of the elements of the set.* For example, the set $\{15, 30, 45, 60, 75, 90\}$ can be described as: $\{n \mid n \text{ is a positive integer divisible by 15 and less than } 100\}$. (The description of the property is called predicate. In this case the set is said to be implicitly specified.)
- [iii] *By recursion.* We define the elements of the set by a computational rule for calculating the elements. For example, the set of all natural numbers leaving a remainder 1 when divided by 3 can be described as

$$\{a_n \mid a_0 = 1, a_{n+1} = a_n + 3\}$$

When the computational rule is clear from the context, we simply specify the set by some initial elements. The previous set can be written as $\{1, 4, 7, 10, \dots\}$. The four elements given suggest the computational rule $a_{n+1} = a_n + 3$.

SUBSETS AND OPERATIONS ON SETS

A set A is said to be a subset of B (written as $A \subseteq B$) if every element of A is also an element of B .

Two sets A and B are equal (we write $A = B$) if their members are the same. In practice, to prove that $A = B$, we prove $A \subseteq B$ and $B \subseteq A$.

A set with no elements is called an empty set, also called a null or a void set and is denoted by $\emptyset$.

Here are some operations on sets.

$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$, called union of A and B .

$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$, called the intersection of A and B .

$A - B = \{x \mid x \in A \text{ and } x \notin B\}$, called the complement of B in A .

A^c denotes $U - A$, where U is the universal set, the set of all elements under consideration.

The set of all subsets of a set A is called the power set of A . it is denoted by 2^A .

Let A and B be two sets. Then $A \times B$ is defined as $\{(a, b) \mid a \in A \text{ and } b \in B\}$. $((a, b)$ is called an ordered pair and is different from (b, a) .)

Definition 1.1 Let S be a set. A collection $\{A_1, A_2, \dots, A_n\}$ of subsets of S is called a partition if $A_i \cap A_j = \emptyset$ ($i \neq j$) and $S = \bigcup_{i=1}^n A_i$ (i.e., $A_1 \cup A_2 \cup \dots \cup A_n$). For example if $S = \{1, 2, 3, \dots, 10\}$ then $\{\{1, 3, 5, 7, 9\}, \{2, 4, 6, 8, 10\}\}$ is a partition of S .

SETS WITH ONE BINARY OPERATION

A binary operation $*$ on a set S is a rule which assigns, to every ordered pair (a, b) of elements from S , a unique element denoted by $a * b$.

Addition for example, is a binary operation on the set Z of all integers.

Union is a binary operation on 2^A , where A is any nonempty set.

Below are five postulates on binary operations.

Postulate 1: Closure. If a and b are in S then $a * b$ is in S .

Postulate 2: Associativity. If a, b, c are in S then $(a * b) * c = a * (b * c)$.

Postulate 3: Identity element. There exists a unique element (called the identity element) e in S such that for any element x in S , $x * e = e * x = x$.

Postulate 4: Inverse. For every element x in S there exists a unique element x' in S such that $x * x' = x' * x = e$. the element x' is called the inverse of x w.r.t $*$.

Postulate 5: Commutativity. If $a, b \in S$ then $a * b = b * a$.

It should be noted that a binary operation may satisfy none of the above five postulates. For example, let $S = \{1, 2, 3, 4, \dots\}$, and let the binary operation be subtraction (i.e. $a * b = a - b$). Closure postulate is not satisfied since $2 - 3 \notin S$. $(2 - 3) - 4 \neq 2 - (3 - 4)$, and so associativity is not satisfied. As we cannot find a positive integer such that $x - e = e - x = x$, postulates 3 and 4 are not satisfied. Obviously $a - b \neq b - a$. therefore, commutativity is not satisfied.

RELATIONS

The concept of relations is a basic concept in computer science as well as in real life. This concept arises when we consider a pair of objects and compare one with the other. For example, “being the father of” gives a relation between two persons. We can express the relation by ordered pairs (for example, “a is the father of b” can be represented by the ordered pair (a, b)).

Definition 1.2 A relation R in a set S is a collection of ordered pairs of elements in S (i.e. a subset of $S \times S$). When (x, y) is in R we write xRy . When (x, y) is not in R we write $xR'y$.

Example A relation R in Z can be defined by xRy if $x > y$.

Properties of Relations

- (i) A relation R in S is reflexive in xRx for every x in S .
- (ii) A relation R in S is symmetric if for x, y in S , yRx whenever xRy .
- (iii) a relation R in S is transitive if for x, y , and z in S , xRz whenever xRy and yRz .

Example A relation R in $\{1, 2, 3, 4, 5, 6\}$ is given by

$$\{(1, 2), (2, 3), (3, 4), (4, 4), (4, 5)\}$$

This relation is not reflexive as $1R'1$. It is not symmetric as $2R3$ but $3R'2$. It is also not transitive as $1R2$ and $2R3$ but $1R'3$.

Definition 1.3 A relation R in a set S is called an equivalence relation if it is reflexive, symmetric and transitive.

Definition 1.4 Let R be an equivalence relation on a set S . Let $a \in S$, then C_a is defined as $\{b \in S | aRb\}$.

C_a is called an equivalence class containing a . in general C_a 's are equivalence classes

CLOSURE OF RELATIONS

A given relation R may not be reflexive or transitive. By adding more ordered pairs to R we can make it reflexive or transitive. For example, consider a relation $R = \{(1,2), (2,3), (1,1), (2,2)\}$ in $\{1, 2, 3\}$. R is not reflexive as $3R3$. But by adding $(3,3)$ to R , we get a reflexive relation. Also, R is not transitive as $1R2$ and $2R3$ but $1R3$. By adding the pair $(1,3)$, we get a relation $T = \{(1,2), (2,3), (1,1), (2,2), (1,3)\}$ which is transitive. There are many transitive relations T containing R . But the smallest among them is interesting.

Definition 1.5 Let R be a relation in a set S . then the transitive closure of R (denoted by R^+) is the smallest transitive relation containing R .

Note: We can define reflexive closure and symmetric closure in a similar way.

Definition 1.6 Let R be a relation in S , then the reflexive transitive closure (denoted by R^*) is the smallest reflexive and transitive relation containing R .

GRAPHS AND TREES

Definition 1.7 a graph (or undirected graph) consists of (a) a nonempty set V called the set of vertices, (b) a set E called the set of edges, and (c) a map Φ which assigns to every edge a unique unordered pair of vertices.

REPRESENTATION OF A GRAPH

Usually a graph, viz, undirected graph, is represented by a diagram where vertices are represented by points or small circles, and edges by arcs joining the vertices of the associated pair (given by the map Φ).

Figure 1 below gives an example of an undirected graph. Thus, the unordered pair $\{v_1, v_2\}$ is associated with the edge e_1 ; the pair $\{v_2, v_2\}$ is associated with

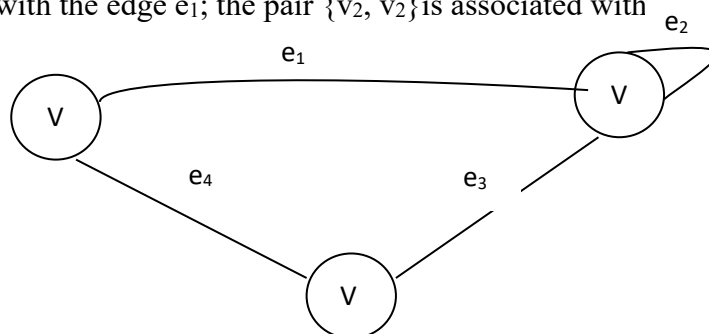


Figure 1 an undirected graph

Definition 1.8 A directed graph (or digraph) consists of (a) a nonempty set V called the set of vertices, (b) a set of E called the set of edges, and (c) a map Φ which assigns to every edge a unique ordered pair of vertices.

Figure 2, for example, gives a directed graph. The ordered pair (v_2, v_3) , (v_3, v_4) , (v_1, v_3) is associated with the edges e_3 , e_4 , e_2 , respectively.

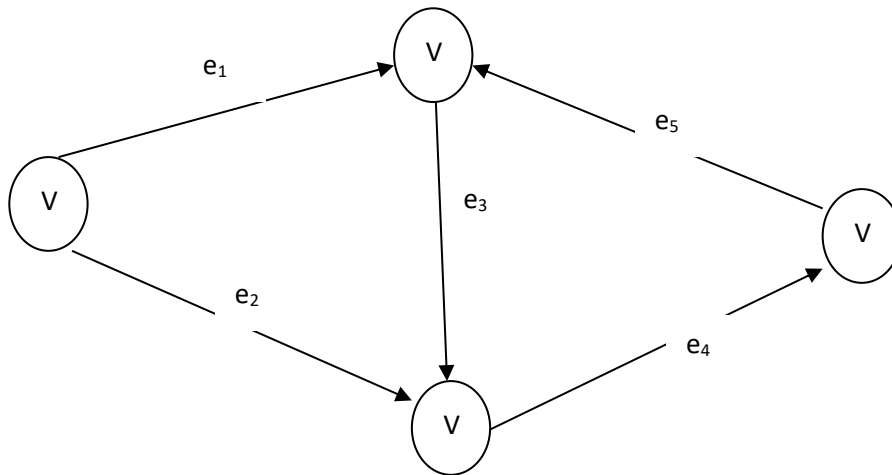


Figure 2 A directed graph

DEFINITIONS

(i) If (v_i, v_j) is associated with an edge e , then v_i and v_j are called vertices of e ; v_i is called a predecessor of v_j which is the successor of v_i .

(ii) if G is a digraph, the undirected graph corresponding to G is the undirected graph obtained by considering the edges and vertices of G , but ignoring the 'direction' of the edges.

Definition 1.9 The degree of a vertex in a graph (directed or undirected) is the number of edges with v as an end vertex. (A self loop is counted twice while calculating the degree).

Definition 1.10. A path in a graph (directed or undirected) is an alternating sequence of vertices and edges of the form $v_1 e_1 v_2 e_2 \dots v_{n-1} e_{n-1} v_n$, beginning and ending with vertices such that e_i has v_i and v_{i+1} as its end vertices and no edge or vertex is repeated in the sequence.

Definition 1.11 A graph (directed or undirected) is connected if there is a path between every pair of vertices.

Definition 1.12 A circuit in a graph is an alternating sequence $v_1 e_1 v_2 e_2 \dots e_{n-1} v_n$ of vertices and edges starting and ending in the same vertex such that e_i has v_i and v_{i+1} as end vertices and no edge or vertex other than v_i is repeated.

TREES

Definition 1.13 A graph (directed or undirected) is called a tree if it is connected and has no circuits.

The graphs given in figures 3 and 4 are examples of trees.

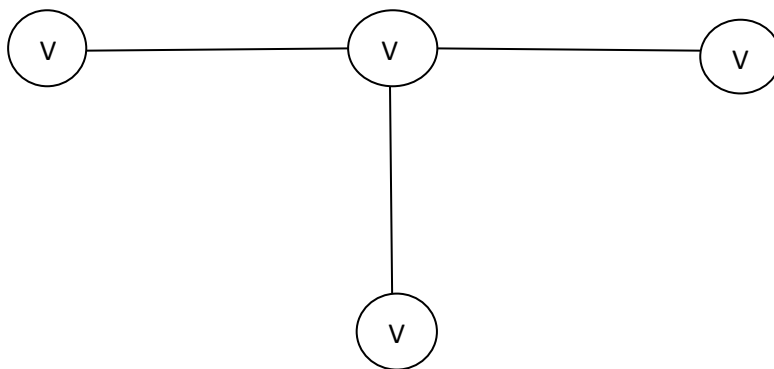


Figure 3 A tree with four vertices

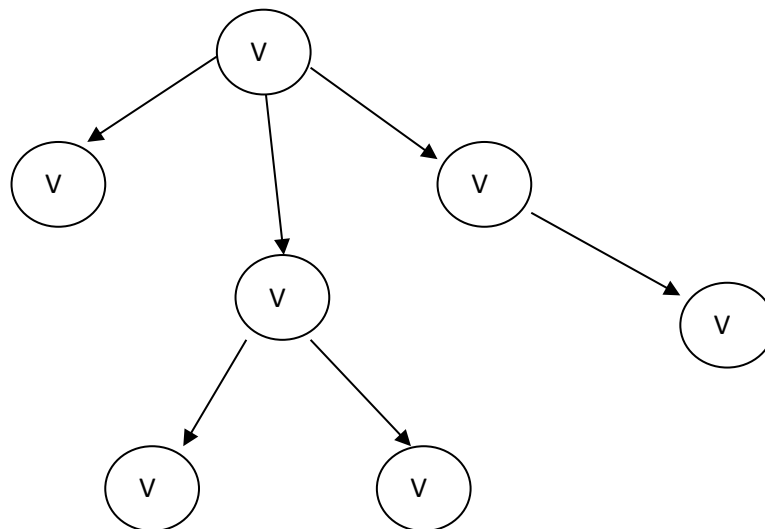


Figure 4 A tree with seven vertices

PROPERTIES OF A TREE

We will discuss some properties of trees (both directed and undirected) used in developing transition systems and studying grammar rules.

Property 1 A tree is a connected graph with no circuits or loops.

Property 2 In a tree there is one and only one path between every pair of vertices.

Property 3 If in a graph there is a unique (i.e. one and only one) path between every pair of vertices then the graph is a tree.

Property 4 A tree with n vertices has $n-1$ edges.

Property 5 if a connected graph with n vertices has $n-1$ edges then it is a tree.

Property 6 If a graph with no circuits has n vertices and $n-1$ edges then it is a tree.

A leaf in a tree can be defined as a vertex of degree one.

COPPERBELT UNIVERSITY
COMPUTER SCIENCE DEPARTMENT
COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)
LECTURE NOTES

UNIT 1.1 Preliminaries (Strings)

Strings of characters are fundamental building blocks in Computer Science. The alphabet over which the strings are defined may vary with the application. For our purposes, we define an **alphabet** to be any finite, nonempty set of symbols. The members of the alphabet are the **symbols** of the alphabet. We generally use capital Greek letter Σ and Γ to designate alphabets and a typewriter font for symbols from an alphabet. The following are a few examples of alphabets.

$$\Sigma_1 = \{0, 1\};$$

$$\Sigma_2 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\};$$

$$\Gamma = \{0, 1, x, y, z\};$$

A **string over an alphabet** is a finite sequence of symbols from the alphabet, usually written next to one another and not separated by commas. In other words, a string (sometimes called a word) is a finite sequence of symbols chosen from some alphabet. If $\Sigma_1 = \{0, 1\}$, then 01001 is a string over Σ_1 . If $\Sigma_2 = \{a, b, c, \dots, z\}$, the, abracadabra is a string over Σ_2 .

Notation: Σ^* denotes the set of all strings (including the empty string Λ) over the alphabet set Σ . $\Sigma^+ = \Sigma^* - \{\Lambda\}$.

Empty String

The empty string is the string with zero occurrences of symbols. The empty string is denoted by ε or Λ .

Length of a string

If w is a string over Σ , the **length** of w , written $|w|$, is the number of symbols that it contains. For example, the string 01101 has length 5 (that is, $|01101| = 5$). The string of length zero is called the empty string and is written ε (or Λ). The empty string plays the role of 0 in a number system. If w has length n , we can write $w = w_1w_2 \dots w_n$ where each $w_i \in \Sigma$.

The **reverse** of w , written w^R , is the string obtained by writing w in the opposite order (i.e., $w_n w_{n-1} \dots w_1$).

Note: In different literature, reverse of a string is known as a transpose of a string and is represented by as w^T where w is the string.

Definition: A palindrome is a string which is the same whether written forward or backward, e.g. Malayalam. A palindrome of even length can be obtained by concatenation of a string and its reverse.

String z is a **substring** of w if z appears consecutively within w . for example, cad is a substring of abracadabra.

Prefix and suffix of a string. A prefix of a string is a substring of leading symbols of that string. For example, the string 123 has 4 prefixes Λ , 1, 12, 123.

Similarly, a suffix of a string is a substring of trailing symbols of that string. For example, the string 123 has suffixes Λ , 3, 23, 123.

Operation on Strings (Concatenation)

The basic operation for strings is binary concatenation operation. We define the operation as follows: let x and y be two strings in Σ^* . Let us form a new string z by placing y after x , i.e. $z = xy$. The string z is said to be obtained by concatenation of x and y . To concatenate a string with itself many times we use the superscript notation

$$\overbrace{xx \dots x}^k = x^k.$$

Example: Find xy and yx where

$$x = 010, \quad y = 1$$

Basic properties of concatenation

Property 1 Concatenation on a set Σ^* is associative since for each x, y, z in Σ^* , $x(yz) = (xy)z$.

Property 2 Identity element The set Σ^* has an identity element Λ w.r.t. the binary operation of concatenation as

$$x\Lambda = \Lambda x = x \quad \text{for every } x \text{ in } \Sigma^*.$$

Property 3 Σ^* has left and right cancellation, for x, y, z in Σ^* .

$$zx = zy \text{ implies } x = y \text{ (left cancellation)}$$

$$xz = yz \text{ implies } x = y \text{ (right cancellation)}$$

Property 4 for x, y in Σ^* , we have

$$|xy| = |x| + |y|.$$

Where $|x|$, $|y|$ and $|xy|$ denote the lengths of the strings x, y, xy , respectively

Lexicographic Ordering of Strings

The lexicographic ordering of strings is the same as the familiar dictionary ordering, except that shorter strings precede longer strings. Thus the lexicographic ordering of all strings over the alphabet $\{0,1\}$ is

$$(\epsilon, 0, 1, 00, 01, 10, 11, 0000).$$

LANGUAGES

A language is a set of strings. More will be said about languages in future lectures.

A set of strings all of which are chosen from some Σ^* where Σ is a particular alphabet, is called a language. If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ . Notice that a language over Σ need not include all strings with all the symbols of Σ , so once we have established that L is a language over Σ . We also know it is a language over any alphabet that is a superset of Σ .

COPPERBELT UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)

LECTURE NOTES

UNIT 2A: FINITE AUTOMATA (DETERMINISTIC FINITE AUTOMATA)

An automaton (plural Automata) is defined as a system where energy, material and information are transformed, transmitted and used for performing some functions without direct participation of man. In Computer Science, the term 'automaton' is defined in a more abstract way as shown in figure 5.

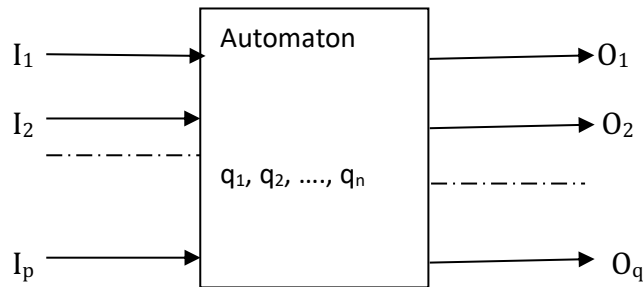


Figure 5 Model of an automaton

An automaton has the following characteristics:

- i. Input. At each of the discrete instants of time $t_1, t_2, \dots$, input value $I_1, I_2, \dots$, each of which can take a finite number of fixed values from the input alphabet Σ , are applied to the input side of the model shown in figure 5.
- ii. Output. $O_1, O_2, \dots, O_q$ are the output of the model, each of which can take finite number of fixed values from an output O .
- iii. States. At any instant of time the automaton can be in one of the states $q_1, q_2, \dots, q_n$.
- iv. State relation. The next state of an automaton at any instant of time is determined by the present state and the present input.
- v. Output relation. Output is related to either state only or to both the input and the state. It should be noted that at any instant of time the automaton is in some state. On 'reading' an input symbol, the automaton moves to the next state which is given by the state relation.

Note: An automaton in which the output depends only on the input is called an automaton without a memory. An automaton in which the output depends on the states also is called

automaton with a finite memory. An automaton in which the output depends only on the states of the machine is called a Moore machine. An automaton in which the output depends on the state and the input at any instant of time is called a Mealy machine.

DESCRIPTION OF A FINITE AUTOMATON

Definition. Analytically, a finite automaton can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- i. Q is a finite nonempty set of states;
- ii. Σ is a finite nonempty set of inputs called input alphabet.
- iii. δ is a function which maps $Q \times \Sigma$ into Q and is usually called direct transition function. This is the function which describes the change of states during the transition. This mapping is usually represented by a transition table or a transition diagram.
- iv. $q_0 \in Q$ is the initial state; and
- v. $F \subseteq Q$ is the set of final states. It is assumed here that there may be more than one final state.

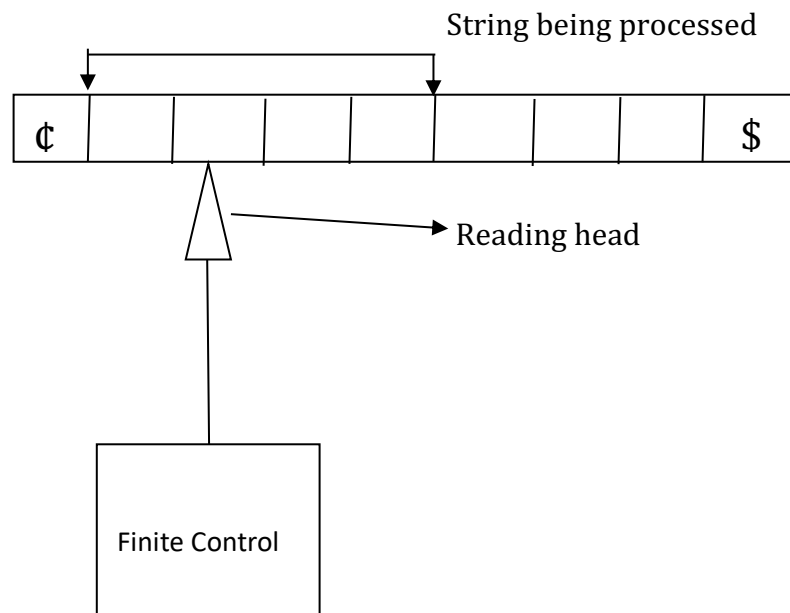


Figure 6 Block diagram of a finite automaton

Figure 6 shows the block diagram for a finite automaton. The various components are explained as follows:

- i. Input tape. The input tape is divided into squares, each square containing a single symbol from the input alphabet Σ . The end squares of the tape contains end markers ϵ at the left

end and \$ at the right end. Absence of end-markers indicates that the tape is of infinite length. The left-to-right sequence of symbols between the end-markers is the input string to be processed.

- ii. Reading Head. The head examines only one square at a time and can move one square either to the left or to the right.
- iii. Finite Control. The input to the finite control will be usually: symbol under the R-head, say a , or the present state of the machine, say q , to give the following outputs: (a) A motion of R-head along the tape to the next square (in some a null move, i.e. R-Head remaining to the same square is permitted); (b) the next state of the finite state machine given by $\delta(q, a)$.

TRANSITION SYSTEMS

A transition graph or a transition system is a finite directed labelled graph in which each vertex (or node) represents a state and the directed edges indicate the transition of a state and the edges are labelled with input/output.

A typical transition system is shown in figure 7. in the figure, the initial state is represented by a circle with an arrow pointing towards it, the final state by two concentric circles,

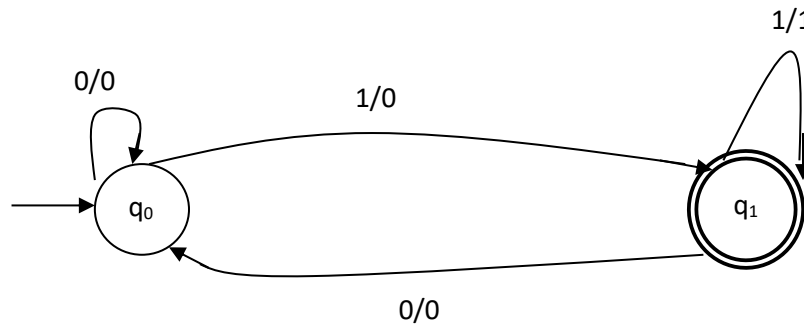


Figure 7 A Transition System

and the other states are represented by just a circle. The edges are labelled by input/output (e.g. by 1/0 or 1/1). For example, if the system is in state q_0 and the input 1 is applied, the system moves to state q_1 as there is a directed edge from q_0 to q_1 with label 1/0. It outputs 0. We will now define a transition system.

Definition: A transition system is a 5-tuple $(Q, \Sigma, \delta, Q_0, F)$ where (a) Q , Σ and F are the finite nonempty set of states, the input alphabet, and the set of final states, respectively, as in the

case of finite automata; (b) $Q_0 \subseteq Q$, and Q_0 is nonempty; and (c) δ is a finite subset of $Q \times \Sigma^* \times Q$. in other word, if (q_1, w, q_2) is in δ , it means the following: the graph starts at the vertex q_1 , goes along a set of edges, and reaches the vertex q_2 . The concatenation of the label along all the edges thus encountered is w .

Definition: A transition system accepts a string w in Σ^* if (a) there exists a path which originates from some initial state, goes along the arrows, and terminates at some final state, and (b) the path value obtained by concatenation of all edge-labels of the path is equal to w .

Example Consider the transition system in figure 8

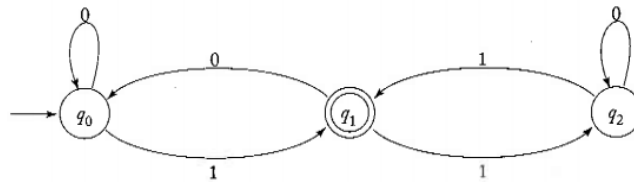


Figure 8

Determine the initial state, final state, and the acceptability of the string 101, 0111 and 1100

Note: Every finite automaton $(Q, \Sigma, \delta, q_0, F)$ can be viewed as a transition system $(Q, \Sigma, \delta', Q_0, F)$ if we take $Q_0 = \{q_0\}$ and $\delta' = \{(q, w, \delta(q, w)) \mid q \in Q, w \in \Sigma^*\}$. But, a transition system need not be a finite automaton. For example, a transition system may contain more than one initial state.

Properties of transition functions

Property 1 $\delta(q, \Lambda) = q$ in a finite automaton. This means the state of the system can be changed only by an input symbol.

Property 2 For all strings w and input symbols a ,

$$\delta(q, aw) = \delta(\delta(q,a),w)$$

$$\delta(q, wa) = \delta(\delta(q,w),a)$$

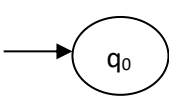
This property gives the state after the automaton consumes or reads the first symbol of a string aw and the state after the automaton consumes a prefix of the string wa .

ACCEPTABILITY OF A STRING BY A FINITE AUTOMATON

Definition: a string x is acceptable by a finite automaton $(Q, \Sigma, \delta, q_0, F)$ if $\delta(q_0, x) = q$ for some $q \in F$. this is basically the acceptability of a string by the final state.

Note: A final state is also called an accepting state.

Example: consider the finite state machine whose transition function δ is given in table in the form of a transition table. Here $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0,1\}$, $F = \{q_0\}$. Give the entire sequence of states for an input string 110001.

Inputs		
State	0	1
 q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Solution:

$$\begin{aligned} \delta(q_0, \overset{\downarrow}{1}10101) &= \delta(q_1, \overset{\downarrow}{1}0101) \\ &\quad \downarrow \\ &= \delta(q_0, \overset{\downarrow}{0}101) \\ &\quad \downarrow \\ &= \delta(q_2, \overset{\downarrow}{1}01) \\ &\quad \downarrow \\ &= \delta(q_3, \overset{\downarrow}{0}1) \\ &\quad \downarrow \\ &= \delta(q_1, 1) \\ &\quad \downarrow \\ &= \delta(q_0, \Lambda) = q_0 \end{aligned}$$

COPPERBELT UNIVERSITY
COMPUTER SCIENCE DEPARTMENT
COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)
LECTURE NOTES

UNIT 2B: FINITE AUTOMATA (NONDETERMINISTIC FINITE AUTOMATA)

Definition: A nondeterministic finite automaton (NFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- i. Q is a finite nonempty set of states;
- ii. Σ is a finite nonempty set of inputs;
- iii. δ is the transition function mapping from $Q \times \Sigma$ into 2^Q which is the power set of Q , the set of all subsets of Q ;
- iv. $q_0 \in Q$ is the initial state; and
- v. $F \subseteq Q$ is the set of final states.

A NFA is defined in the same way as a DFA except that multiple-state transitions and ϵ -transitions are allowed.

A multi-state transition means that at each move, there could be more than one next state. That is, for any state q and any input symbol a , the value of $\delta(q, a)$ is a subset of Q , where $\delta(q, a) = \{p_1, p_2, \dots, p_k\}$ means that the next state from state q , after reading a can be any one of $p_1, p_2, \dots, p_k$. A special case is that $\delta(q, a)$ could be the empty set $\emptyset$. This means that the machine has no next state and hangs, and the input is rejected regardless of what remaining input symbols are. It is the equivalent of going to a failure state in a DFA.

In addition to multiple-state transitions, we also allow ϵ -transitions (or, ϵ -moves) in an NFA. An ϵ -transition is a move in which the tape head does not do anything (it neither reads nor moves), but the state can be changed. In other words, we allow a transition like $\delta(q, a) = \{p_1, \dots, p_k\}$, which means that state q can be changed to any one of $p_1, p_2, \dots$, or p_k , without reading a symbol from the input.

Therefore, the transition function δ is, formally, a function of the form

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q,$$

where 2^Q denotes the collection of all subsets of Q . The following is an example of a transition function of an NFA M :

Inputs		
State	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_2
q_1		$\{q_1, q_2\}$
α		

NFA's, like DFA's, can also be represented by transition diagrams. In the transition diagram, we will still use a vertex to represent a state and a labelled edge to represent a move, except that we allow multiple edges from one vertex to other vertices with the same label. That is, if $\delta(q, \alpha) = \{p_1, p_2, \dots, p_k\}$, then we draw k edges from state q to each of $p_1, p_2, \dots, p_k$, and each with a label α . For instance, the transition diagram of the transition function M above is as shown in figure 9.

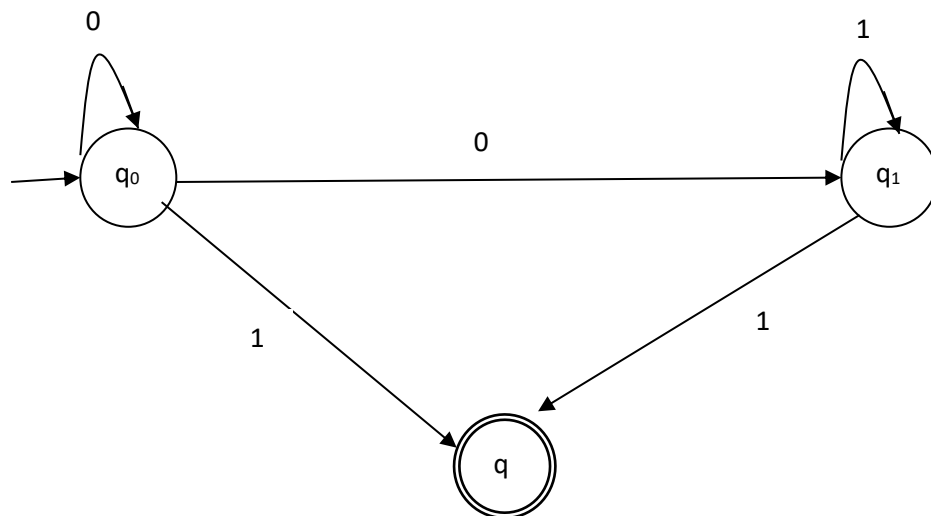
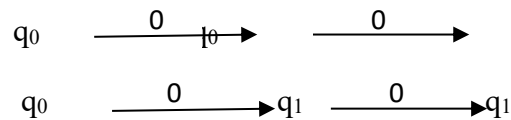


Figure 9 Transition diagram of an NFA

On an input x , an NFA may have more than one computation path. For instance, for M shown in figure 9, the input 00 has the following computation paths:



we note that the differences between the deterministic and nondeterministic automata is only in δ . For deterministic automaton the outcome is a state, i.e. an element of Q ; for nondeterministic automaton the outcome is a subset of Q .

ACCEPTABILITY OF A STRING BY AN NDFA

Definition: A string $w \in \Sigma^*$ is accepted by NDFA M if $\delta(q_0, w)$ contains some final state.

Note: As M is nondeterministic, $\delta(q_0, w)$ may have more than one state. So w is accepted by M if a final state is one among the possible states M can reach on application of w .

Definition: the set acceptable by an automaton (deterministic or nondeterministic) is the set of all input strings accepted by M . it is denoted by $T(M)$.

WHY NONDETERMINISM?

In reasoning about nondeterministic machines, we should be quite cautious in using intuitive notions. Intuition can easily lead us astray, and we must be able to give precise arguments to substantiate our conclusions. Nondeterminism is a difficult concept. Digital computers are completely deterministic; their state at any time is uniquely predictable from the input and the initial state. Thus it is natural to ask why we study nondeterministic machines at all. We are trying to model real systems, so why include such nonmechanical features as choice?

Let us consider a game-playing program to explain why we might need to consider non-determinism.

Many deterministic algorithms require that one make a choice at some stage. A typical example is a game-playing program. Frequently, the best move is not known, but can be found using an exhaustive search with backtracking. When several alternatives are possible, we choose one and follow it until it becomes clear whether or not it was best. If not, we retreat to the last decision point and explore the other choices. A nondeterministic algorithm that can make the best choice would be able to solve the problem without backtracking, but a deterministic one can simulate nondeterminism with some extra work. For this reason, nondeterministic machines can serve as models of search-and-backtrack algorithms.

COPPERBELT UNIVERSITY
COMPUTER SCIENCE DEPARTMENT
COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)
LECTURE NOTES
UNIT 2B: FINITE AUTOMATA (THE EQUIVALENCE OF DFA AND NDFA)

Although it is easier to construct, a non-deterministic is just an idealized machine which cannot be efficiently implemented in practice, since a real machine can only follow one computation path at a time. In this lecture, we will look at the way of finding out the equivalence of DFA and NDFA.

Definition: Two finite automata (acceptors) M_1 and M_2 are said to be equivalent if

$$L(M_1) = L(M_2),$$

That is, if they both accept the same language.

There are generally many acceptors for a given language, so any DFA or NDFA has many equivalent acceptors.

Theory: For every NDFA, there exists a DFA which simulates the behaviour of NDFA. Alternatively, if L is the set accepted by NDFA, then there exists a DFA which also accepts L .

PROOF Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NDFA accepting L . We construct a DFA M' as follows:

$$M' = (Q', \Sigma, \delta, q'_0, F')$$

Where

- (i) $Q' = 2^Q$ (any state in Q' is denoted by $[q_1, q_2, \dots, q_j]$, where $q_1, q_2, \dots, q_j \in Q$);
- (ii) $q'_0 = [q_0]$;
- (iii) F' is the set of all subsets of Q containing an element of F .

Before defining δ' , let us look at the construction of Q' , q'_0 and F' . M is initially at q_0 . But on application of an input symbol, say a , M can reach any of the states in $\delta(q_0, a)$. To describe M , just after the application of the input symbol a , we require all the possible states that M can reach

after the application of a . So, M' has to remember all these possible states at any instant of time. hence the states of M' are defined as subsets of Q . as M starts with initial state q_0 , q'_0 is defined as $[q_0]$. A string w belongs to $T(M)$ if a final state is one of the possible states M reaches on processing w . So a final state in M' (i.e. an element of F') is any subset of Q containing some final state of M .

Now we can define δ' :

$$\delta'([q_1, q_2, \dots, q_i], a) = \delta(q_1, a) \cup \delta(q_2, a) \dots \cup \delta(q_i, a).$$

Equivalently, $\delta'([q_1, q_2, \dots, q_i], a) = [p_1 \dots p_j]$ if and only if

$$\delta'([q_1, \dots, q_i], a) = [p_1, p_2, \dots, p_j]$$

before proving $L = T(M')$, we prove an auxilliary result:

$$\delta'(q'_0, x) = [q_1, \dots, q_i]$$

if and only if $\delta(q_0, x) = [q_1, \dots, q_i]$ for all x in Σ^* (1)

we prove by induction on $|x|$, the “if” part, i.e.

$$\delta'(q'_0, x) = [q_1, q_2, \dots, q_i], \text{ if } \delta(q_0, x) = [q_1, \dots, q_i] \quad (2)$$

when $|x| = 0$, $\delta(q_0, \Lambda) = q'_0 = [q_0]$. So (2) is true for x with $|x| = 0$. Thus there is a basis for induction.

Assume (2) is true for all input strings y with $|y| \leq m$. Let x be a string of length $m+1$. We can write x as ya . where $|y| = m$ and $a \in \Sigma$. Let $\delta(q_0, y) = [q_1, \dots, q_i]$ and $\delta(q_0, ya) = \{r_1, r_2, \dots, r_k\}$. As $|y| \leq m$ by induction hypothesis we have

$$\delta'(q'_0, y) = [q_1, \dots, q_i] \quad (3)$$

also $\{r_1, r_2, \dots, r_k\} = \delta(q_0, ya) = \delta(\delta(q_0, y), a) = \delta([q_1, \dots, q_i], a)$. By definition of δ'

$$\delta'([q_1, \dots, q_i], a) = [r_1, \dots, r_k] \quad (4)$$

Hence

$$\begin{aligned} \delta'(q'_0, ya) &= \delta'(\delta'(q'_0, y), a) = \delta'([q_1, \dots, q_i], a) \text{ by (3)} \\ &= [r_1, \dots, r_k] \text{ by (4).} \end{aligned}$$

Thus we have proved (2) for $x = ya$.

By induction (2) is true for all strings x . the other part (i.e. “only if”, part) can be proved , similarly and so (1) is established.

Now $x \in T(M)$ if and only if $\delta(q, x)$ contains a state of F . By (1), $\delta(q_0, x)$ contains a state of F if and only if $\delta'(q'_0, x)$ is in F' . Hence $x \in T(M)$ if and only if $x \in T(M')$. this proves that DFA M' accepts L .

Note: In the construction of a deterministic finite automaton M_1 , the only difficult is the construction δ' for M_1 . By definition, $\delta'([q_1, \dots, q_k], a) = \bigcup_{i=1}^k \delta(q_i, a)$. So we have to apply δ to (q_i, a) for each $i = 1, 2, \dots, k$ and take the union to get $\delta'([q_1, \dots, q_k], a)$.

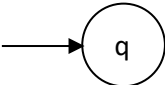
When δ is given in terms of a state table (i.e. transition table), the construction is simpler. $\delta(q_i, a)$ is given by the row corresponding to q_i and column corresponding to a . To construct $\delta'([q_1, \dots, q_k], a)$, consider the states appearing in the rows corresponding to $q_1, \dots, q_k$, and the column corresponding to a . These states constitute $\delta'([q_1, \dots, q_k], a)$.

Note: we write δ' as δ itself when there is no ambiguity, we also mark the initial state with $\rightarrow$ and the final state with circle in the state table.

Example 1

Construct a deterministic automaton equivalent to $M = ([q_0, q_1], [0, 1], \delta, q_0, \{q_0\})$. δ is given by its state table (table 1)

Table 1 State Table for Example 1

State / Σ	0	1
 q_0	q_0	q_1
q_1	q_1	q_0, q_1

SOLUTION For the deterministic automaton M_1 ,

- The states are subsets of $[q_0, q_1]$, i.e. $\emptyset, [q_0], [q_1], [q_0, q_1]$;
- $[q_0]$ is the initial state;
- $[q_0]$ and $[q_0, q_1]$ are the final states as these are the only states containing q_0 , and
- δ is given by the state table given in table 2.

Table 2 state table of M_1

State/ Σ	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$[q_0]$	$[q_0]$	$[q_1]$
$[q_1]$	$[q_1]$	$[q_0, q_1]$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_1]$

q_0 and q_1 appear in the rows corresponding to q_0 and q_1 and the column corresponding to 0. So, $\delta([q_0, q_1], 0) = [q_0, q_1]$.

When M has n states, the corresponding deterministic finite automaton has 2^n states. However, we need not construct δ for all these 2^n states, but only for those states reachable from $[q_0]$. This is because our interest is only in constructing M_1 accepting $T(M)$. so we start the construction of δ for $[q_0]$. We continue by considering only states appearing earlier under input columns and constructing δ for these states. We halt when there is no more new states appearing under the input columns.

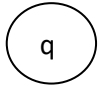
Example 2

Find the deterministic acceptor equivalent to

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$$

δ is given in table 3.

Table 3 State table for example2

States/ Σ	a	b
$\rightarrow q_0$	q_0, q_1	q_2
q_1	q_0	q_1
		q_0, q_1

Solution

The deterministic automaton M_1 equivalent to M is defined as follows:

$$M_1 = \{2^Q, \{a, b\}, \delta, [q_0], F\}$$

Where

$$F = \{[q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2]\}$$

We start the construction by considering $[q_0]$ first. We get $[q_2]$ and $[q_0, q_1]$. Then we construct δ for $[q_2]$ and $[q_0, q_1]$. $[q_1, q_2]$ is a new state appearing under the input columns. After constructing the δ for $q_1, q_2]$, we don't get any new states and so we terminate the construction of δ . The state table is given in table 4.

State/ Σ	a	B
$[q_0]$	$[q_0, q_1]$	$[q_2]$
$[q_2]$	$\emptyset$	$[q_0, q_1]$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_1, q_2]$
$[q_1, q_2]$	$[q_0]$	$[q_0, q_1]$

COPPERBELT UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)

LECTURE NOTES

UNIT 2C: (MINIMISATION OF FINITE AUTOMATA)

Any DFA defines a unique language, but the converse is not true. For a given language, there are many DFA's that accept it. There may be a considerable difference in the number of states of such automata. For example, The two DFA's depicted in Figure 10 are equivalent as a few test strings will quickly reveal. We notice some obviously unnecessary features of (a) in figure. The state q_5 plays absolutely no role in the automaton since it can never be reached from the initial state q_0 . Such a state is inaccessible, and it can be removed (along with all transitions relating to it) without affecting the language accepted by the automaton. But even after the removal of q_5 , the first automaton has some redundant parts. The states reachable subsequent to the first move $\delta(q_0, 0)$ mirror those reachable from a first move $\delta(q_0, 1)$. The second automaton combines these two options.

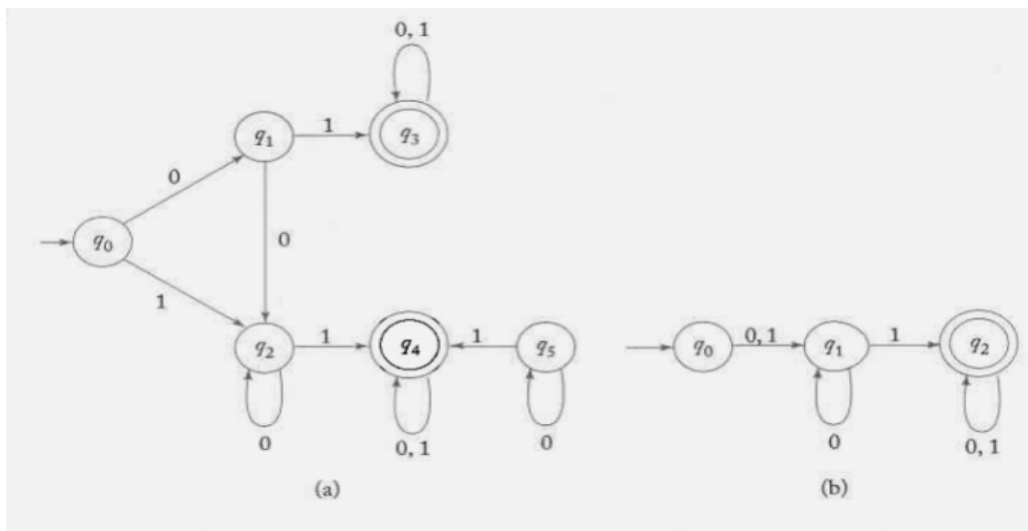


Figure 10 Example of Equivalent automata

From a strictly theoretical point of view, there is little reason for preferring the automaton in (b) over that in (a). However in terms of simplicity, the second alternative is clearly preferable. Representation of an automaton for the purpose of computation requires space

proportional to the number of states. For storage efficiency, it is desirable to reduce the number of states as far as possible. We now describe an algorithm that accomplishes this.

Definition 1 Two states q_1 and q_2 are equivalent (denoted by $q_1 \equiv q_2$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are nonfinal states for all $x \in \Sigma^*$.

As it is difficult to construct $\delta(q_1, x)$ and $\delta(q_2, x)$ for all $x \in \Sigma^*$, we give one more definition.

Definition 2 Two states q_1 and q_2 are k -equivalent ($k \geq 0$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both nonfinal states for all strings x of length k or less. In particular, any two final states are 0-equivalent and any two nonfinal states are also 0-equivalent.

Here are some of the properties of these relations.

Property 1 The relations we have defined, i.e. equivalence and k -equivalence, are equivalence relations, i.e. they are reflexive, symmetric and transitive.

Property 2 By using the theory that “any equivalence relation R on a set S partitions S into equivalence classes”, these induce partitions of Q . These partitions can be denoted by π and π_k , respectively. Elements of π_k are k -equivalence classes.

Property 3 if q_1 and q_2 are k -equivalent for all $k \geq 0$, they are equivalent.

Property 4 If q_1 and q_2 are $(k+1)$ -equivalent, then they are k -equivalent.

Property 5 $\pi_n = \pi_{n+1}$ for some n . (π_n denotes the set equivalence classes under n -equivalence.)

The following result is key to the construction of minimum state automaton.

RESULT Two states q_1 and q_2 are $(k+1)$ equivalent if (a) they are k -equivalent; (b) $\delta(q_1, a)$ and $\delta(q_2, a)$ are also k -equivalent for every $a \in \Sigma$.

Construction of minimum automaton

Step 1 (Construction of π_0). By definition of 0-equivalence, $\pi_0 = (Q_1^0, Q_2^0)$, where Q_1^0 is the set of all final states and $Q_2^0 = Q - Q_1^0$.

Step 2 (Construction of π_{k+1} from π_k). Let Q_i^k be any subset of π_k . If q_1 and q_2 are in Q_i^k , they are $(k+1)$ -equivalent provided $\delta(q_1, a)$ and $\delta(q_2, a)$ are k -equivalent. Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are in the same equivalence class in π_k for every $a \in \Sigma$. If so q_1 and q_2 are $(k+1)$ equivalent. In this way Q_i^k is further divided into $(k+1)$ -equivalence classes. Repeat this for every Q_i^k in π_k to get all the elements of π_{k+1} .

Step 3 Construct π_n for $n = 1, 2, \dots$ until $\pi_n = \pi_{n+1}$.

Step 4 (Construction of minimum automaton). For the required minimum state automaton, the states are the equivalence classes obtained in step 3, i.e. the elements of π_n . the state table is obtained by replacing a state q by the corresponding equivalence class $\{q\}$.

Note: The construction of π_0, π_1, π_2 etc, is easy when the transition table is given. $\pi_0 = (Q_1^0, Q_2^0)$, where $Q_1^0 = F$ and $Q_2^0 = Q - F$. the subsets in π_1 are obtained by partitioning subsets of π_0 further. If q_1 and $q_2 \in Q_1^0$, consider the states in each a -column, where $a \in \Sigma$ corresponding to q_1 and q_2 . If they are in the same subset of π_0 , q_1 and q_2 1-equivalent. In general, $(k+1)$ -equivalent states are obtained by applying the above method for q_1 and q_2 in Q_i^k .

Example: Construct a minimum state automaton equivalent to the following automaton given in figure11.

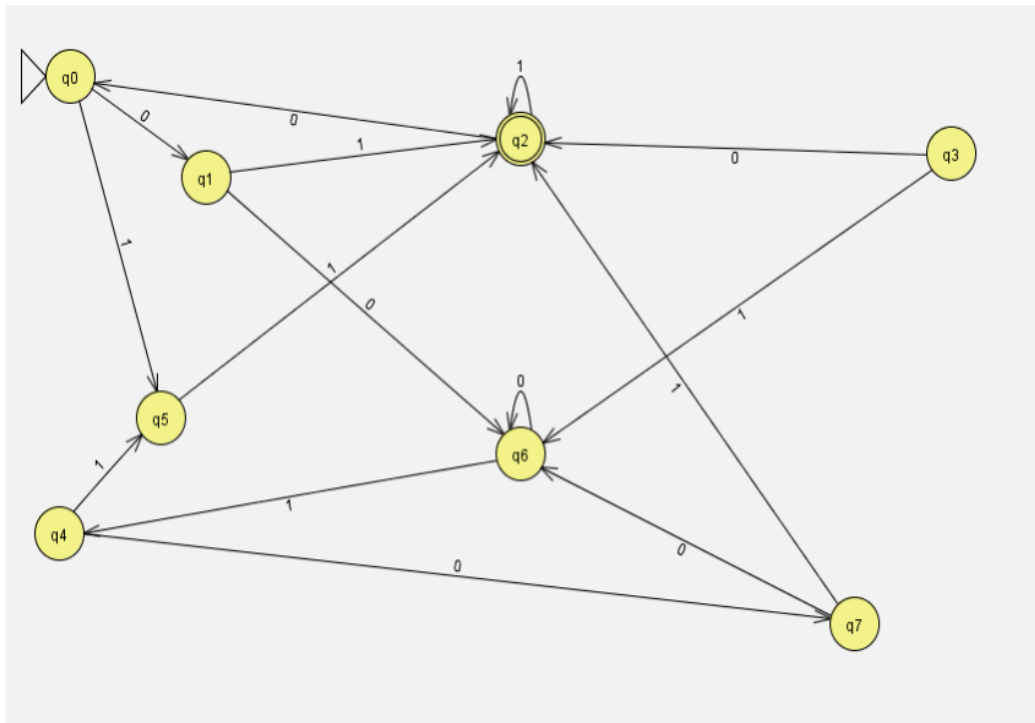


Figure 11 DFA of example

Solution

It will be easier if we construct the transition table given in table 4.

Table 4 transition table for the example

State / Σ	0	1
$\rightarrow q_0$	q_1	q_5
q_1	q_6	q_2
q	q_0	q_2
q_3	q_2	q_6
q_4	q_7	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

By applying step 1, we get

$$Q_1^0 = F = \{q_2\} \text{ and } Q_2^0 = Q - F$$

So

$$\pi_0 = \{\{q_2\}, \{q_0, q_1, q_3, q_4, q_5, q_6, q_7\}\}$$

$\{q_2\}$ in π_0 cannot be further partitioned. So $Q_1' = \{q_2\}$. Consider q_0 and $q_1 \in Q_1^0$. The entries under 0-column corresponding to q_0 and q_1 are q_1 and q_6 ; they lie in Q_2^0 . The entries under 1-column are q_5 and q_2 . $q_2 \in Q_1^0$ and $q_5 \in Q_2^0$. Therefore, q_0 and q_1 are not 1-equivalent. Similarly, q_0 is not 1-equivalent to q_3, q_5 , and q_7 .

Now consider q_0 and q_4 . The entries under 0-column are q_1 and q_7 . Both lie in Q_2^0 . The entries under 1-column are q_5, q_5 . So q_4 and q_0 are 1-equivalent. Similarly, q_0 is 1-equivalent to q_6 . $\{q_0, q_4, q_6\}$ is a subset in π_1 . So, $Q_2' = \{q_0, q_4, q_6\}$.

Repeat the construction by considering q_1 and any one of the states q_3, q_5 and q_7 . q_1 is not 1-equivalent to q_3 or q_5 but is 1-equivalent to q_7 . Hence, $Q'_3 = \{q_1, q_7\}$. The elements left over in Q_2^0 are q_3 and q_5 . By considering the entries under 0-column and 1-column, we see that q_3 and q_5 are 1-equivalent. So $Q'_4 = \{q_3, q_5\}$. Therefore

$$\pi_1 = \{\{q_2\}, \{q_0, q_4, q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

$\{q_2\}$ is also in π_2 as it cannot be partitioned further. Now the entries under 0-column corresponding to q_0 and q_4 lie in the same equivalence class in π_1 . The entries under 1-column are q_5, q_5 . So q_0 and q_4 are 2-equivalent. Hence $\{q_0, q_4, q_6\}$ is partitioned into $\{q_0, q_4\}$ and $\{q_6\}$. q_1 and q_7 are 2-equivalent. q_3 and q_5 also 2-equivalent. Thus

$$\pi_2 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}.$$

q_0 and q_4 are 3-equivalent. q_1 and q_7 are 3-equivalent. q_3 and q_5 are 3-equivalent. Therefore

$$\pi_3 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}.$$

As $\pi_2 = \pi_3$, π_2 gives the equivalence class. The minimum state automaton is

$$M' = (Q', \{0,1\}, \delta', q'_0, F')$$

Where

$$Q' = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\},$$

$$q'_0 = \{q_0, q_4\}, \quad F' = \{q_2\}$$

And δ' is given by table 5.

Table 5 Transition table of minimum state automaton

State/ Σ	0	1
$[q_0, q_4]$	$[q_1, q_7]$	$[q_3, q_5]$
$[q_1, q_7]$	$[q_6]$	$[q_2]$
$[q_2]$	$[q_0, q_4]$	$[q_2]$
$[q_3, q_5]$	$[q_2]$	$[q_6]$
$[q_6]$	$[q_6]$	$[q_0, q_4]$

Note

The transition diagram for the minimum state automaton is figure 13. The state q_0 and q_4 are identified and treated as one state. (So also is q_1, q_7 and q_3, q_5 .) but the transitions in both the diagrams (figure 11 and 12) are the same.

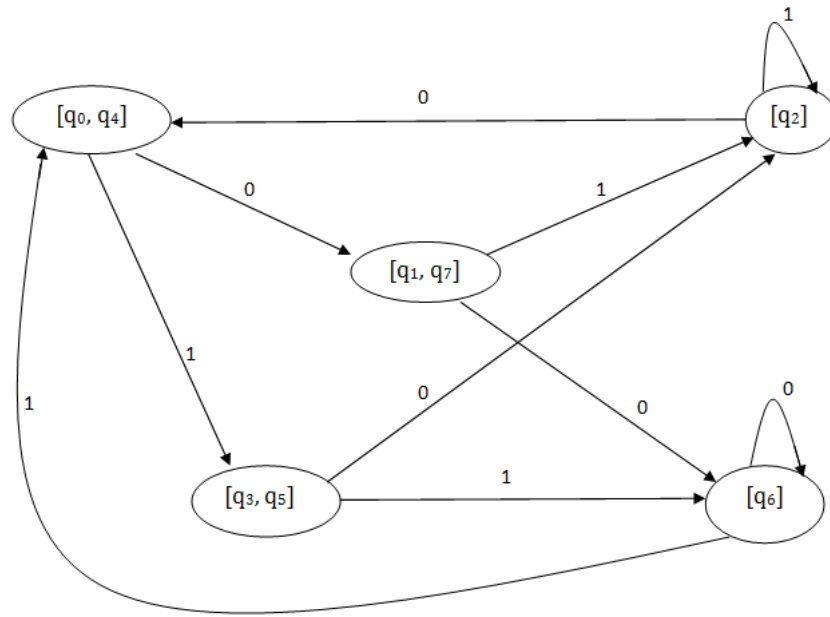


Figure 12 Minimum State automaton

COPPERBELT UNIVERSITY
COMPUTER SCIENCE DEPARTMENT
COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)
LECTURE NOTES

UNIT 2C: (FINITE AUTOMATA WITH OUTPUTS)

The automata we have been looking at so far have two outputs, i.e. they accept the string or do not accept the string. This acceptability was decided on the basis of reachability of the final state by the initial state. Now, we remove this restriction and consider the model where the outputs can be chosen from some other alphabet. The value of the output function $Z(t)$ in the most general case is a function of the present state $q(t)$ and the present input $x(t)$, i.e.

$$Z(t) = \lambda (q(t), x(t))$$

Where λ is called the output function. This generalized model is usually called Mealy machine. If the output function $Z(t)$ depends only on the present state and is independent of the current input, the output function may be written as

$$Z(t) = \lambda (q(t))$$

This restricted model is called Moore machine. It is more convenient to use Moore machine in automata theory.

Definition of the Moore Machine

The Moore machine is a six-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where

- (i) Q is a finite set of states;
- (ii) Σ is the input alphabet;
- (iii) Δ is the output alphabet
- (iv) δ is the transition function $\Sigma \times Q$ into Q ;
- (v) λ is the output function mapping Q into Δ ; and
- (vi) q_0 is the initial state.

Definition of the Mealy machine

A mealy machine is a six-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where all the symbols except λ have the same meaning as in the Moore machine. λ is the output function mapping $\Sigma \times Q$ into Δ .

Table gives a Moore machine. The initial state q_0 is marked with an arrow. The table defines δ and λ .

Present State	Next State		Output Λ
	$a = 0$	$a = 1$	
q_0	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

For the input string 0111 the transition of states is given by $q_0 \rightarrow q_3 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$. The output string is

COPPERBELT UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)

LECTURE NOTES

UNIT 8: FORMAL LANGUAGES

Introduction

There are two kinds of languages, namely natural languages and formal languages. Before we look at Formal languages, we will briefly look at natural languages. The phrase ‘Natural language’ refers to languages like English, Bemba, etc. They are called natural because we do not (consciously) invent them; we naturally acquire them. Some properties of natural language are generally understood to fall into three areas: syntax (or grammar), semantics, and pragmatics.

Syntax (grammar)

‘Syntax’ refers to the way in which words may be combined to form phrases and sentences. In other words, the permissible ways in which words can be combined is syntax. By following the syntax, we are able to generate new sentences.

Semantics

The semantics of a natural language such as English is an exceptionally complex and confusing area. Just what is part of semantics as opposed to syntax or pragmatics is a question much disputed. One common understanding is that semantics is concerned with the relation between language and the ‘world’. Hence, the meaning of a sentence determines the conditions under which it is true.

Pragmatics

‘Pragmatics’ is used as a general term for referring to all kinds of effects on what we say - the thought we express - given the context of our utterance. One way of understanding pragmatics is as an ‘extra element’ on top of syntax and semantics. The latter two determine the ‘literal meaning’ of a sentence; given a literal meaning, pragmatics covers how we can express a different or extended meaning, on the basis of the ‘literal meaning’ + context. Detailed are some common aspects of language use that fall under pragmatics.

(i) Implicature

An implicature is where a speaker says something with a literal meaning, but intends her audience to infer that she means something else on the basis of shared background information. Here are some examples.

(1) Master to servant: “It’s cold in here”

Implicature: Close the window.

Background: Both master and servant are in a room with a draught caused by an open window.

(2) X to Y in a restaurant: Is anyone sitting here?

Background: the chair is empty.

(ii) Missing constituents

(1) a. It’s raining (where?)

a. Bill and Mary are married (to each other or not?)

b. Fixing the fault will take time (Sure, but how long?)

c. There is nothing on TV tonight (worth watching?)

FORMAL LANGUAGES

A formal language is one we invent for some purpose or other. A programming language such as Java, for instance, is a formal language invented for the purpose of programming computers. We will motivate our discussion with examples from natural languages, where grammatical rules are used to distinguish those strings of symbols that constitute acceptable sentences from those that do not. For example, we might describe a simple English sentence as a structure with a subject followed by a predicate. The subject might take the form of a single noun, while the predicate might be an intransitive verb or perhaps a transitive verb followed by an object. This top-down decomposition could continue until we reached the level of detail in which we would describe the literal symbols that actually appear in the English sentences. A transitive verb has two characteristics. First, it is an action verb, expressing a doable activity like kick, want, paint, write, eat, clean, etc. Second, it must have a direct object, something or someone who receives the action of the verb. When no direct object follows an action verb, the verb is intransitive. Such a decomposition is illustrated in figure 13 below.

<Sentence> → <subject><predicate><period>

<Subject> → <noun>

<Noun> → John

<Noun> → Mary

<Predicate> → <intransitive verb>

<Predicate> → <transitive verb> <object>

<Intransitive verb> → skates

<Transitive verb> → hit

<Transitive verb> → likes

<Object> → <noun>

<Period> → .

Figure 13 A grammar describing a small subset of the English language

We need to note the following regarding the listing above. First, we will note that some terms are enclosed in angular bracket while others are not. Terms that do not appear in brackets are called **terminals**. These are symbols that may actually appear in a sentence. The terms enclosed in brackets are called **nonterminals**. One of these nonterminals (in figure 13, it is <sentence>) is considered the **start symbol**. Each line figure 13, consisting of a left and right side connected by an arrow is called a **rewrite rule**. The right side of each rule represents a more detailed description of the left side. For instance, the first rewrite rule of figure 13 indicates that a <sentence> has the structure of a <subject> followed by a <predicate> followed by a <period>. We can see that figure 13 describes the structure of a sentence in a hierarchical manner, beginning with the start symbol <sentence>.

Such a finite collection of nonterminals and terminals together with a start symbol and a finite set of rewrite rules is called a **grammar**, or more precisely a **phase-structure grammar**, since it based on the composition of strings in terms of “phrases” with each phrase represented by a nonterminal. More formally, a grammar is defined as a quadruple (V_N, Σ, P, S) , where V_N is a finite set of nonterminals, Σ is a finite set of terminals, S (an element of V) is the start symbol, and P is a finite set of rewrite rules or production rules. In general, the left and right sides of a grammar’s rewrite rules can be any combination of terminals and nonterminals as long as the left

side contains at least one nonterminal. Moreover, the right side of some rewrite rules may consist of the empty string. Such a rule is called a **λ -rule**.

The use of brackets to distinguish nonterminals from terminals helps prevent confusion. For example, in a grammar describing the structure of English sentences, the term sentence might appear both as a nonterminal representing the start symbol and as a terminal representing a word in a sentence. In this case the use of brackets allows us to distinguish between dual uses of the term. In our study, we will use a less cumbersome notational system. Unless otherwise specified, we will denote nonterminals by upper case letters and terminals by lower case letters. Thus a rule of the form $S \rightarrow xN$ will mean that the nonterminal S can be refined as the terminal x followed by the nonterminal N .

A grammar is said to generate a particular string of terminals if, by starting with the start symbol, one can produce that string by successively replacing patterns found on the left of the grammar's rewriting rules with the corresponding expressions on the right, until only terminals remain. The sequence of steps in this process is called a **derivation** of the string. For example, the string (the sentence)

Mary likes John.

can be generated from the grammar in figure 13 through the derivation in figure 14.

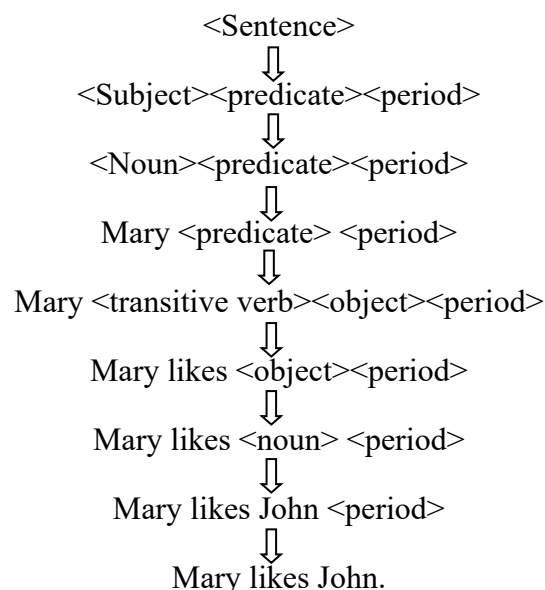


Figure 14 A derivation of the sentence "Mary likes John."

The listing in figure 15 below shows another grammar with start symbol S, and a derivation showing how that grammar can generate the string xyz.

$S \rightarrow XSZ$
$S \rightarrow Y$
$Y \rightarrow yZ$
$Y \rightarrow \lambda$
$X \rightarrow x$
$Z \rightarrow z$

$S \Rightarrow XSZ \Rightarrow xSZ \Rightarrow xYZ \Rightarrow xyYZ \Rightarrow xyZ \Rightarrow xyz$
--

Figure 15 A grammar and an associated derivation of the string xyz

If the terminals of a grammar G are symbols in an alphabet Σ , we say that G is a grammar over the alphabet Σ . In such a case the strings generated by G are actually strings in Σ^* . Thus, a grammar G over an alphabet Σ specifies a language over Σ that consist of the strings G generates. This language is denoted $L(G)$.

We now define a **regular grammar** as a grammar whose rewrite rules conform to the following restrictions. The left side of any rewrite rule in a regular grammar must consist of a single nonterminal, and the right side must be a terminal followed by a nonterminal, or a single terminal, or the empty string. Thus, rewrite rules of the form

$$Z \rightarrow yX$$

$$Z \rightarrow x$$

$$W \rightarrow \lambda$$

would be allowed in a regular grammar, whereas rules of the form

$$yW \rightarrow X$$

$$X \rightarrow xZy$$

$$YX \rightarrow WvZ$$

would not.

Common notation used with formal languages and grammar

- a) If A is any set, then A^* denotes the set of all strings over A . A^+ denotes $A^* - \{\Lambda\}$. Where Λ is the empty string.
- b) $A, B, C, A_1, A_2 \dots$ denote variables.
- c) $a, b, c, \dots$ denotes terminals.
- d) $x, y, z, w, \dots$ denotes strings of terminal.
- e) α, β, Γ denotes elements of $(V_N \cup \Sigma)^*$.
- f) $X^0 = \Lambda$ for any symbol X in $V_N \cup \Sigma$.

Derivations and the language generated by a Grammar

Productions are used to derive one string over $V_N \cup \Sigma$ from another string. The formal definition of derivations is as follows:

Definition: if $\alpha \rightarrow \beta$ is a production in a grammar G and γ, δ are any two strings on $V_N \cup \Sigma$ then we say $\gamma\alpha\delta$ directly derives $\gamma\beta\delta$ in G (we write $\gamma\alpha\delta \xRightarrow{G} \gamma\beta\delta$). This process is called one-step derivation. In particular, if $\alpha \rightarrow \beta$ is a production, then $\alpha \xRightarrow{G} \beta$.

NOTE: If α is a part of a string and $\alpha \rightarrow \beta$ is a production, we can replace α by β in that string (without altering the remaining parts). In this case, we say the string we started with directly derives the new string.

For example $G = (\{S\}, \{0,1\}, \{S \rightarrow 0S1, S \rightarrow 01\}, S)$

has the production $S \rightarrow 0S1$. So, S in 0^4S1^4 can be replaced by $0S1$. The resulting string 0^40S11^4 . Thus we have $0^4S1^4 \xRightarrow{G} 0^40S11^4$.

Remarks on Derivation

- Any derivation involves the application of productions. When the number of times we apply productions is one, we write $\alpha \xRightarrow{G} \beta$; when it is more than one time we write (

$$\alpha \xRightarrow{*G} \beta$$

- The string generated by the most recent application of production is called the working string.
- The derivation of a string is complete when the working string cannot be modified. If the final string does not contain any variable, it is a sentence in the language. If the final string contains a variable, it is a sentential form and in this case the production generator gets stuck.

We give several examples of grammars and languages generated by them.

Example: If $G = (\{S\}, \{0,1\}, \{S \rightarrow 0S1, S \rightarrow \Lambda\}, S)$, find $L(G)$ the language generated by G .

Solution:

As $S \rightarrow \Lambda$ is a production, $S \Rightarrow_G \Lambda$. So Λ is in $L(G)$. Also, for all $n \geq 1$,

$$S \xRightarrow[G]{\quad} 0S1 \xRightarrow[G]{\quad} 0^2S1^2 \xRightarrow[G]{\quad} \dots \xRightarrow[G]{\quad} 0^nS1^n \xRightarrow[G]{\quad} 0^n1^n$$

Therefore,

$$0^n1^n \in L(G) \text{ for } n \geq 0$$

(Note that in the above derivation $S \rightarrow 0S1$ is applied at every step except in the last step. In the last step we apply $S \rightarrow \Lambda$). Hence, $\{0^n1^n \mid n \geq 0\} \subseteq L(G)$.

To show that $L(G) \subseteq \{0^n1^n \mid n \geq 0\}$, we start with w in $L(G)$. the derivation of w starts with S . if $S \rightarrow \Lambda$ is applied first, we get Λ . In this case $w = \Lambda$. Otherwise the first production to be applied is $S \rightarrow 0S1$. At any stage if we apply $S \rightarrow \Lambda$, we get a terminal string. also, the terminal string is obtained only by applying $S \rightarrow \Lambda$. Thus the derivation of w is of the form

$$S \xRightarrow[G]{*} 0^nS1^n \xRightarrow[G]{\quad} 0^n1^n$$

i.e.

$$L(G) \subseteq \{0^n1^n \mid n \geq 0\}.$$

Therefore,

$$L(G) = \{0^n1^n \mid n \geq 0\}$$

Example: If $G = (\{S\}, \{a\}, \{S \rightarrow SS\}, S)$, find the language generated by G .

Solution: $L(G) = \emptyset$, since there only production $S \rightarrow SS$ in G has no terminal on the right-hand side.

Significance of regular grammars

The significance of regular grammars is that the languages they generate are exactly the languages that are recognized by finite automata.

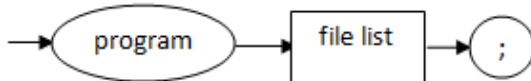
The grammatical approach is very significant to language specification as we are going to see in future lectures, where we will see that the use of grammars leads to a classification scheme for various types of languages. For now we will compare the composition of grammars to that of the syntax diagrams that are often used when describing the syntax of programming languages such as Pascal or Ada. We will find that these syntax diagrams are essentially another way of representing the rewrite rules of a grammar (see figure 16).

SYNTAX DIAGRAM

Program



Header



File list



REWRITE RULES

$\langle \text{program} \rangle \rightarrow \langle \text{header} \rangle \langle \text{block} \rangle .$

$\langle \text{header} \rangle \rightarrow \text{program } \langle \text{file list} \rangle ;$

$\langle \text{file list} \rangle \rightarrow (\langle \text{identifier list} \rangle);$

Figure 16 Some Syntax diagrams describing parts of the Pascal programming language as compared to their equivalent grammatical forms

Regular expressions

Regular expressions are an algebraic way to define patterns. In other words, a regular expression is a string that is used to describe or match a set of strings according to certain syntax rules.

Operands of Regular Expressions

Like all algebras, regular expressions have certain kinds of atomic operands. In the algebra of arithmetic, atomic operands are constants (e.g., integers or reals) or variables whose possible values are constants. In the algebra of regular expressions, an atomic operand is one of the following:

1. A character from the alphabet over which the regular expression is defined,
2. The symbol ϵ ,
3. The symbol $\emptyset$, or
4. A variable whose value can be any pattern defined by a regular expression.

Values of Regular Expressions

Expressions in any algebra have values of a certain type. For arithmetic expressions, values are integers, reals, or whatever type of numbers with which we are working.

For regular expressions, the value of each expression is a pattern consisting of a set of strings often called a language. The language denoted by a regular expression E will be referred to as $L(E)$, or the or the “language of E .” The languages of the atomic operands are defined as follows.

1. If x is any character, then the regular expression x stands for the language $\{x\}$; that is, $L(x) = \{x\}$. Note that this language is a set that contains one string; the string has length 1, and the lone position of that string has the character x .
2. $L(\epsilon) = \{\epsilon\}$. The special symbol ϵ as a regular expression denotes the set whose only string is the empty string, or string of length 0.
3. $L(\emptyset) = \emptyset$. The special symbol $\emptyset$ as a regular expression denotes the empty set of strings.

Note that we do not define a value for an atomic operand that is a variable. Such an operand only takes on a value when we replace the variable by a concrete expression, and its value is then whatever value that expression has.

Operators of Regular Expressions

There are three operators used in regular expressions. These can be grouped using parentheses, as in the algebras with which we are familiar. There is an order of precedence and associativity laws that allow us to omit some pairs of parentheses, just as we do in arithmetic expressions. We shall describe the rules regarding parentheses after we examine the operators.

1. Union

The first, and most familiar, is the union operator, which we shall denote $|$ or $+$. The rule for union is that if R and S are two regular expressions, then $R | S$ denotes the union of the languages that R and S denote. That is, $L(R | S) = L(R) \cup L(S)$. Recall that $L(R)$ and $L(S)$ are each sets of strings, so the notion of taking their union makes sense.

Example: If a is a regular expression denoting $\{a\}$, and b is a regular expression denoting $\{b\}$. Thus $a | b$ is a regular expression denoting $\{a, b\}$. That is the set containing the two strings a and b , each string being of length 1.

2. Concatenation

The second operator for the algebra of regular expressions is called concatenation. It is represented by no operator symbol at all, just as multiplication is sometimes written without an operator; for instance, in arithmetic ab denotes the product of a and b . Like union, concatenation is a binary, infix operator. If R and S are regular expressions, then RS is the concatenation of R and S . $L(RS)$, the language denoted by RS , is formed from the languages $L(R)$ and $L(S)$, as follows. For each string r in $L(R)$ and each string s in $L(S)$, the string rs , the concatenation of strings r and s , is in $L(RS)$. Recall the concatenation of two lists, such as character strings, is formed by taking the elements of the first list, in order, and following them by the elements of the second list, in order.

Example: Let R be the regular expression a , and so $L(R)$ is the set $\{a\}$. Also let S be the regular expression b , so $L(S) = \{b\}$. Then RS is the expression ab . To form $L(RS)$, we need to take every string in $L(R)$ and concatenate it with every string in $L(S)$. In this simple case, both languages $L(R)$ and $L(S)$ are singletons, so we have only one choice from each. We pick a from $L(R)$ and b from $L(S)$, and concatenate these lists of length 1 to get the string ab . Thus $L(RS)$ is $\{ab\}$.

3. Closure

The third operator is called Kleene closure or just closure. It is a unary, postfix operator; that is, it takes one operand and it appears after that operand. Closure is denoted by a star, so R^* is the closure of regular expression R . Because the closure operator is of highest precedence, it is often necessary to put parentheses around the R , and write $(R)^*$.

The effect of the closure operator is to say “zero or more occurrences of strings in R .” That is, $L(R^*)$ consists of

1. The empty string ϵ , which we can think of as zero occurrences of strings in R .
2. All the strings in $L(R)$; these represent one occurrence of a string in $L(R)$.
3. All the strings in $L(RR)$, the concatenation of $L(R)$ with itself; these represent two occurrences of strings in $L(R)$.
4. All the strings in $L(RRR)$, $L(RRRR)$, and so on, representing three, four, and more occurrences of strings in $L(R)$.

We can informally write

$$R^* = \epsilon \mid R \mid RR \mid RRR \mid \dots$$

However, we must understand that the expression on the right side of the equals sign is not a regular expression, because it contains an infinite number of occurrences of the union operator. All regular expressions are built from a finite number of occurrences of the three operators.

Example: Let $R = a$. What is $L(R^*)$? Surely ϵ is in this language, as it must be in any closure. The string a , which is the only string in $L(R)$, is in the language, as is aa from $L(RR)$, aaa from $L(RRR)$, and so on. That is, $L(a^*)$ is the set of strings of zero or more a 's, or $\{\epsilon, a, aa, aaa, \dots\}$.

Precedence of Regular Expression Operators

As we have mentioned informally above, there is a conventional order of precedence for the three operators union, concatenation, and closure. This order is

1. Closure (highest), then
2. Concatenation, then
3. Union (lowest).

Thus when interpreting any expression, we first group the closure operators by finding the shortest expression immediately to the left of a given $*$ that has the form of an expression (i.e., parentheses, if any, are balanced). We may place parentheses around this expression and the $*$.

Next, we may consider the concatenation operators, from the left. For each, we find the smallest expression immediately to the left and the smallest expression immediately to the right, and we put parentheses around this pair of expressions. Finally, we consider union operators from the left. We find the smallest expression immediately to the left and right of each union, and we place parentheses around this pair of expressions with the union symbol in the middle.

Examples

The set of strings over $\{0,1\}$ that end in 3 consecutive 1's.

$(0 \mid 1)^* 111$

The set of strings over $\{0,1\}$ that have at least one 1.

$0^* 1 (0 \mid 1)^*$

The set of strings over $\{0,1\}$ that have at most one 1.

$0^* \mid 0^* 1 0^*$

Applications of regular expressions

Regular expressions are very useful in Programming languages and search engines. For example, the sorts of things regular expressions can do in Perl include:

- Making sure your user has entered the data you're looking for—input validation
- Verifying that input is in the right specific format, for example, that email addresses have the right components
- Extracting parts of a file that match a specific criteria (for example, you could extract the headings from a file to build a table of contents, or extract all the links in an HTML file).
- Splitting a string into elements based on different separator fields (and often, complex nested separator fields)
- Finding irregularities in a set of data—multiple spaces that don't belong there, duplicated words, errors in formatting
- Counting the number of occurrences of a pattern in a string
- Searching and replacing—find a string that matches a pattern and replace it with some other string

COPPERBELT UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 345)

LECTURE NOTES

UNIT 9: CONTEXT-FREE GRAMMAR AND PUSHDOWN AUTOMATA

INTRODUCTION

In this lecture we present context-free grammars, a more powerful method of describing languages. Such grammars can describe certain features that have a recursive structure, which makes them useful in a variety of applications.

Context-free grammars were first used in the study of human languages. One way of understanding the relationship of terms such as noun, verb, and preposition and their respective phrases leads to a natural recursion because noun phrases may appear inside verb phrases and vice versa. Context-free grammars can capture important aspects of these relationships.

An important application of context-free grammars occurs in the specification and compilation of programming languages. A grammar for a programming language often appears as a reference for people trying to learn the language syntax. Designers of compilers and interpreters for programming languages often start by obtaining a grammar for the language. Most compilers and interpreters contain a component called a parser that extracts the meaning of a program prior to generating the compiled code or performing the interpreted execution. A number of methodologies facilitate the construction of a parser once a context-free grammar is available. Some tools even automatically generate the parser from the grammar.

The collection of languages associated with context-free grammars are called the context-free languages. They include all the regular languages and many additional languages. In this lecture, we give a formal definition of context-free grammars and study the properties of context-free languages. We also introduce pushdown automata, a class of machines recognizing the context-free languages. Pushdown automata are useful because they allow us to gain additional insight into the power of context-free grammars.

CONTEXT-FREE GRAMMARS (CFG)

The following is an example of a context-free grammar, which we will call G1.

$$A \rightarrow OA1$$

$$A \rightarrow B$$

$$B \rightarrow \#.$$

As we defined earlier, a grammar consists of a collection of substitution rules, also called productions. Each rule appears as a line in the grammar, comprising a symbol and a string separated by an arrow. The symbol is called a variable (or nonterminal). The string consists of variables and other symbols called terminals. The variable symbols often are represented by capital letters. The terminals are analogous to the input alphabet and often are represented by lowercase letters, numbers, or special symbols. One variable is designated as the start variable. It usually occurs on the left-hand side of the topmost rule. For example, grammar G1 contains three rules. G1's variables are A and B, where A is the start variable. Its terminals are 0, 1, and #. You use a grammar to describe a language by generating each string of that language in the following manner.

1. Write down the start variable. It is the variable on the left-hand side of the top rule, unless specified otherwise.
2. Find a variable that is written down and a rule that starts with that variable. Replace the written down variable with the right-hand side of that rule.
3. Repeat step 2 until no variables remain. For example, grammar G1 generates the string 000#111. The sequence of substitutions to obtain a string is called a derivation. A derivation of string 000#111 in grammar G1 is

$$A \Rightarrow OA1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

You may also represent the same information pictorially with a derivation (parse) tree. An example of a parse tree is shown in Figure 17. It is easy to visualize derivations in context-free languages as we can represent derivations using tree structures.

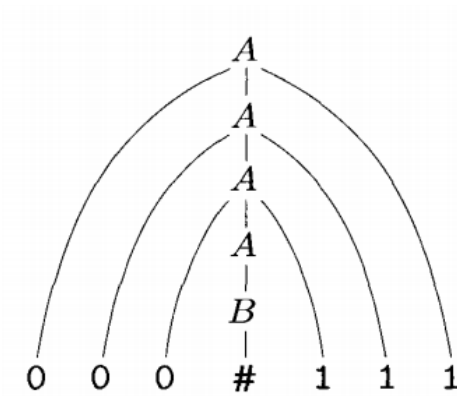


Figure 17: Parse tree for 000#111 in grammar G1

DERIVATION TREES

The derivation in a CFG can be represented using trees. Such tree representing derivations are called derivation trees. We give a rigorous definition of a derivation tree.

Definition: a derivation tree (also called a parse tree) for a CFG $G = (V_N, \Sigma, P, S)$ is a tree satisfying the following:

- (i) Every vertex has a label which is a variable or terminal or Λ .
- (ii) The root has label S .
- (iii) The label of an internal vertex is a variable.
- (iv) If the vertex $n_1, n_2, \dots, n_k$ written with labels $X_1, X_2, \dots, X_k$ are the sons of vertex n with label A , then $A \rightarrow X_1 X_2 \dots X_k$ is a production in P .
- (v) A vertex n is a leaf if its label is $a \in \Sigma$ or Λ ; n is the only son of its father if its label is Λ .

For example, let $G = (\{S, A\}, \{a, b\}, P, S)$, where P consist of $S \rightarrow aAS \mid a \mid SS$, $A \rightarrow SbA \mid ba$. Figure is an example of a derivation tree.

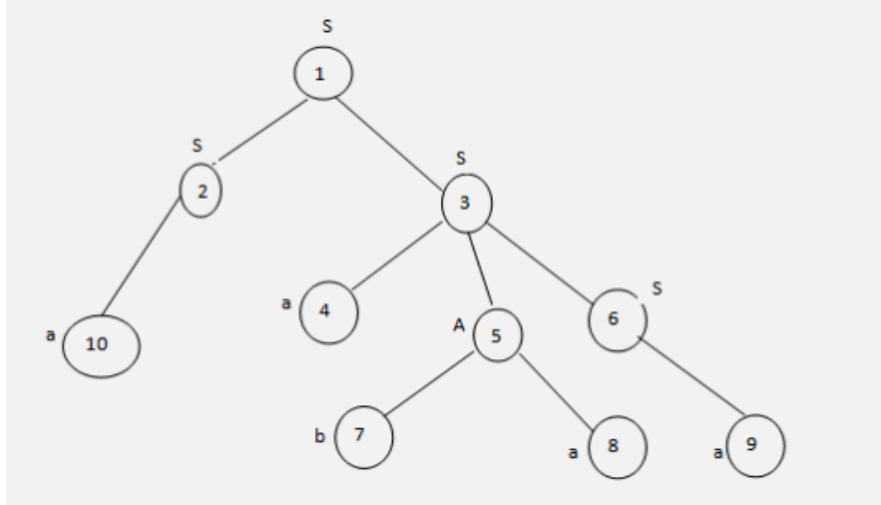


Figure 18 Derivation Tree

Note: Vertices 4-6 are sons of 3 written from the left, and $S \rightarrow aAS$ is in P . Vertices 7 and 8 also sons of 5 written from the left, and $A \rightarrow ba$ is a production in P . Vertex 5 is an internal vertex and its label is A , which is a variable.

Ordering of leaves from the Left

We can order all the vertices of a tree in the following way: the successors of the root (i.e. sons of the root) are ordered from the left. So in figure 18, the vertices at level 1 are ordered from the left. If v_1 and v_2 are two vertices at level 1 and v_1 is to the left of v_2 then we say that v_1 is to the left of any son of v_2 . Thus we get a left-to-right ordering of vertices at level 2. Repeating the process up to level k , where k is the height of the tree, we have an ordering of all vertices from the left.

Definition: The yielding of a derivation tree is the concatenation of the labels of the leaves without repetition in the left-to-right ordering.

The yielding of the derivation tree in of figure 18 is aabaa.

This derivation is obtained using the sequence below

$$S \Rightarrow SS \Rightarrow aS \Rightarrow aaAS \Rightarrow aabaS \Rightarrow aabaa$$

Definition: A subtree of a derivation tree T is a tree (a) whose root is vertex v of T , (b) whose vertices are the descendants of v together with their labels, (c) whose edges are those connecting the descendants of v . Figure 19 shows a subtree of the derivation tree shown in figure 18.

Note: A subtree looks like a derivation tree except that the label of the root may not be S .

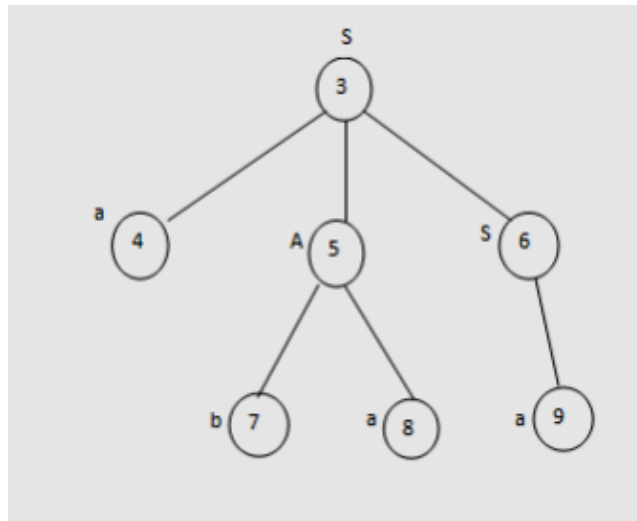


Figure 19 Subtree of figure 18

Regular Grammar vs Context-free grammar

Every regular grammar is context-free, but not all context-free grammars are regular. The following context-free grammar, however, is also regular.

$$S \rightarrow a$$

$$S \rightarrow aS$$

$$S \rightarrow bS$$

The terminals here are a and b , while the only non-terminal is S . The language described is all nonempty strings of a 's and b 's that end in a . This grammar is regular: no rule has more than one nonterminal in its right-hand side, and each of these nonterminals is at the same end of the right-hand side.

More on Parse Trees

Every terminal string generated by a grammar has a corresponding parse tree; every valid parse tree represents a string generated by the grammar (called the *yield* of the parse tree).

Example: Given the following grammar, find a parse tree for the string $1 + 2 * 3$:

1. $\langle E \rangle \rightarrow \langle D \rangle$
2. $\langle E \rangle \rightarrow (\langle E \rangle)$
3. $\langle E \rangle \rightarrow \langle E \rangle + \langle E \rangle$
4. $\langle E \rangle \rightarrow \langle E \rangle - \langle E \rangle$
5. $\langle E \rangle \rightarrow \langle E \rangle * \langle E \rangle$
6. $\langle E \rangle \rightarrow \langle E \rangle / \langle E \rangle$
7. $\langle D \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots 9$

The parse tree is:

$$\begin{array}{c} E \rightarrow E \rightarrow N \rightarrow 1 \\ + \\ E \rightarrow E \rightarrow N \rightarrow 2 \\ * \\ E \rightarrow N \rightarrow 3 \end{array}$$

AMBIGUOUS GRAMMARS

A grammar for which there are two different parse trees for the same terminal string is said to be *ambiguous*.

The grammar for balanced parentheses given earlier is an example of an ambiguous grammar:

$$P \rightarrow (P) \mid P P \mid \text{epsilon}$$

We can prove this grammar is ambiguous by demonstrating two parse trees for the same terminal string.

Here are two parse trees for the empty string:

```
P --> P --> epsilon
      P --> epsilon

P --> epsilon
```

Here are two parse trees for ():

```
P --> P --> (
      P --> epsilon
      )
      P --> epsilon

P --> P --> epsilon
      P --> (
              P --> epsilon
              )
```

While in general it may be difficult to prove a grammar is ambiguous, the demonstration of two distinct parse trees for the same terminal string is sufficient proof that a grammar is ambiguous.

An unambiguous grammar for the same language (that is, the set of strings consisting of balanced parentheses) is:

```
P --> ( P ) P | epsilon
```

THE PROBLEM OF AMBIGUOUS GRAMMARS

A parse tree is supposed to display the structure used by a grammar to generate an input string. This structure is not unique if the grammar is ambiguous. A problem arises if we attempt to impart meaning to an input string using a parse tree; if the parse tree is not unique, then the string has multiple meanings.

We typically use a grammar to define the syntax of a programming language. The structure of the parse tree produced by the grammar imparts some meaning on the strings of the language.

If the grammar is ambiguous, the compiler has no way to determine which of two meanings to use. Thus, the code produced by the compiler is not fully determined by the program input to the compiler.

PUSHDOWN AUTOMATA

In this lecture we introduce a new type of computational model called pushdown automata. These automata are like nondeterministic finite automata but have an extra component called a stack. The stack provides additional memory beyond the finite amount available in the control. The stack allows pushdown automata to recognize some non-regular languages.

Pushdown automata are equivalent in power to context-free grammars. This equivalence is useful because it gives us two options for proving that a language is context free. We can give either a context-free grammar generating it or a pushdown automaton recognizing it. Certain languages are more easily described in terms of generators, whereas others are more easily described in terms of recognizers.

Figure 20 is a schematic representation of a finite automaton. The control represents the states and transition function, the tape contains the input string, and the arrow represents the input head, pointing at the next input symbol to be read.

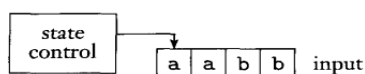


Figure 20: Finite State Automaton illustration

With the addition of a stack component we obtain a schematic representation of a pushdown automaton, as shown in figure 21.

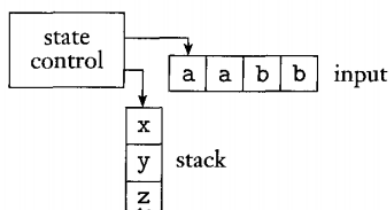


Figure 21 Pushdown Automaton Illustration

A pushdown automaton (PDA) can write symbols on the stack and read them back later. Writing a symbol "pushes down" all the other symbols on the stack. At any time the symbol on the top of the stack can be read and removed. The remaining symbols then move back up. Writing a symbol on the stack is often referred to as pushing the symbol, and removing a symbol is referred to as popping it. Note that all access to the stack, for both reading and writing, may be done only at the top. In other words a stack is a "last in, first out" storage device. If certain

information is written on the stack and additional information is written afterward, the earlier information becomes inaccessible until the later information is removed.

Plates on a cafeteria serving counter illustrate a stack. The stack of plates rests on a spring so that when a new plate is placed on top of the stack, the plates below it move down. The stack on a pushdown automaton is like a stack of plates, with each plate having a symbol written on it. A stack is valuable because it can hold an unlimited amount of information.

Recall that a finite automaton is unable to recognize the language with very large strings because of its finite memory. A PDA is able to recognize language with large strings because it can use its stack to store the parts of the string it has seen. Thus the unlimited nature of a stack allows the PDA to store numbers of unbounded size. The following informal description shows how the automaton for a language comprising of 0s and 1s works.

Read symbols from the input. As each 0 is read, push it onto the stack. As soon as 1 is seen, pop a 0 off the stack for each 1 read. If reading the input is finished exactly when the stack becomes empty of 0s, accept the input. If the stack becomes empty while there is still some input from the alphabet then reject the input.

pushdown automata may be nondeterministic. Deterministic and nondeterministic pushdown automata are not equivalent in power. Nondeterministic pushdown automata recognize certain languages which no deterministic pushdown automata can recognize, though we will not prove this fact. Recall that deterministic and nondeterministic finite automata do recognize the same class of languages, so the pushdown automata situation is different. We focus on nondeterministic pushdown automata because these automata are equivalent in power to context-free grammars.

FORMAL DEFINITION OF A PUSHDOWN AUTOMATON

The formal definition of a pushdown automaton is similar to that of a finite automaton, except for the stack. The stack is a device containing symbols drawn from some alphabet. The machine may use different alphabets for its input and its stack, so now we specify both an input alphabet Σ and a stack alphabet Γ .

At the heart of any formal definition of an automaton is the transition function, which describes its behaviour. Recall that $\Sigma^* = \Sigma \cup \{ \epsilon \}$ and $\Gamma^* = \Gamma \cup \{ \epsilon \}$. The domain of the transition function is $Q \times \Sigma^* \times \Gamma^*$. Thus the current state, next input symbol read, and top symbol of the stack determine the next move of a pushdown automaton. Either symbol may be ϵ , causing the machine to move without reading a symbol from the input or without reading a symbol from the stack.

For the range of the transition function we need to consider what to allow the automaton to do when it is in a particular situation. It may enter some new state and possibly write a symbol on

the top of the stack. The function δ can indicate this action by returning a member of Q together with a member of Γ^* , that is, a member of $Q \times \Gamma^*$. Because we allow nondeterminism in this model, a situation may have several legal next moves. The transition function incorporates nondeterminism in the usual way, by returning a set of members of $Q \times \Gamma^*$, that is, a member of $P(Q \times \Gamma^*)$. Putting it all together, our transition function δ takes the form $\delta: Q \times \Sigma^* \times \Gamma^* \rightarrow P(Q \times \Gamma^*)$.

DEFINITION

A PDA is formally defined as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$ where

- Q is a finite set of *states*
- Σ is a finite set which is called the *input alphabet*
- Γ is a finite set which is called the *stack alphabet*
- δ is a finite subset of $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma^* \rightarrow P(Q \times \Gamma^*)$, the *transition relation*.
- q_0 is the *start state*
- $Z \in \Gamma$ is the *initial stack symbol*
- F is the set of *accepting states*

Notions of Acceptance:

There are two different notions of acceptance of a string by a PDA. According to the first, a PDA accepts a string when, after reading the entire string, the PDA is in a final state. According to the second, a PDA accepts a string when, after reading the entire string, the PDA has emptied its stack.

Acceptability By Final state

A PDA accepts a string w by final state if and only if, for some $q_f \in F$, the set of final states of the PDA, $(q_0, w, z_0) \rightarrow (q_f, \epsilon, g)$. For any given PDA, our default assumption will be that of acceptance by final state. The language of the PDA will be defined as follows:

$$\{w \mid \exists g. (q_0, w, z_0) \rightarrow (q_f, \epsilon, g) \text{ for } q_f \in F\}.$$

Acceptance by empty stack

To further highlight PDAs that accept by empty stack, we leave out the F component from their seven-tuple presentation, thus obtaining the six-tuple $P_2 = (Q, \Sigma, \Gamma, \delta, q_0, z_0)$. For such PDAs, a string w is in its language exactly when the following is true: $(q_0, w, z_0) \rightarrow q, \epsilon, \epsilon$.

Here, $q \in Q$, i.e., q is any state. All that matters is that the input is entirely consumed and an empty stack results in doing so.

EXAMPLES OF PUSHDOWN AUTOMATA

The following is the formal description of the PDA that recognizes the language $\{0^n 1^n \mid n \geq 0\}$. Let M_1 be $(Q, \Sigma, \Gamma, \delta, q_1, \epsilon, F)$, where

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table.

Input:	0			1			ϵ		
Stack:	0	\$	ϵ	0	\$	ϵ	0	\$	ϵ
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$			$\{(q_3, \epsilon)\}$			
q_3						$\{(q_3, \epsilon)\}$			$\{(q_4, \epsilon)\}$
q_4									

We can also use a state diagram to describe a PDA, as shown in figure 22.

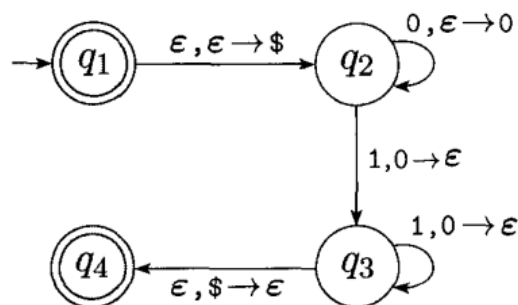


Figure 22

Note: In most text if the stack is initially empty, the empty symbol (ϵ) is not shown. In such cases the definition of the PDA is given as a six-tuple excluding the initial stack symbol.

Example

Consider the PDA $M = (\{q, p\}, \{a, b, c\}, \{a, b\}, \delta, q, \{p\})$,

where δ is defined as follows:

$$\delta(q, a, \epsilon) = \{(q, a)\}, \quad \delta(p, a, a) = \{(p, \epsilon)\},$$

$$\delta(q, b, \epsilon) = \{(q, b)\}, \quad \delta(p, b, b) = \{(p, \epsilon)\},$$

$$\delta(q, c, \epsilon) = \{(p, \epsilon)\}.$$

This machine works in the following way: At the initial state q , it pushes the input symbols a and b to the stack until it reads a tape symbol c . After reading c , it moves to state p . In state p , M compares each symbol on the tape with the top symbol of the stack. If they are all equal, then the machine accepts the input.

Determine whether the string $aabbcbb$ is acceptable by the PDA. What is the symbol that will be on top of the stack after the input string?

PUSHDOWN AUTOMATA AND CONTEXT-FREE LANGUAGES

It is possible to prove that the sets accepted by PDA (by empty stack or final state) are precisely context-free languages.

Theorem: If L is a context-free language, then we can construct a PDA A accepting L by empty store (empty stack), i.e. $L = N(A)$.

Proof: We construct A by making use of productions in G .

Step 1: (Construction of A) Let $L = L(G)$, where $G = (V_N, \Sigma, P, S)$ is a context-free grammar. We construct a PDA A as

$$A = (\{q\}, \Sigma, V_N \cup \Sigma, \delta, q, S, \emptyset)$$

Where δ is defined by the following rules:

$$R_1: \delta(q, \Lambda, A) = \{(q, a) \mid A \rightarrow a \text{ is in } P\}$$

$$R_2: \delta(q, a, a) = \{(q, \epsilon)\} \quad \text{for every } a \text{ in } \Sigma$$

We can explain the construction in the following way: the Pushdown symbols in A are variables and terminals. If the PDA reads a variable A on top of the pushdown store (PDS), it makes a Λ -move by replacing the right-hand side of any A -production (after erasing A). If the pda reads a

terminal a on PDS and if it matches with the current input symbol, then the PDA erases a . In other cases the pda halts.

If $w \in L(G)$ is obtained by a left-most derivation

$$S \Rightarrow u_1 A_1 a_1 \Rightarrow u_1 u_2 A_2 a_2 a_1 \Rightarrow \dots \Rightarrow w,$$

Then A can empty the PDS on application of input string w . The first move of A is by a Λ -move corresponding to $S \rightarrow u_1 A_1 a_1$. The PDA erases S and stores $u_1 A_1 a_1$. Then using rule R_2 , the PDA erases the symbols in u_1 by processing a prefix of w . Now, the topmost symbol in PDS is A_1 . Once again by applying the Λ -move corresponding to $A_1 \rightarrow u_2 A_2 a_2$, the pda erases A_2 and stores $u_2 A_2 a_2$ above a_1 . Proceeding in this way, the pda empties the PDS by processing the entire string w .

Example

We'll apply the following context-free grammar: $S \rightarrow 0BB$, $B \rightarrow 0S \mid 1S \mid 0$. Test whether 010^4 is in $N(A)$.

Solution: Define pda A as follows:

$$A = (\{q\}, \{0,1\}, \{S,B,0,1\}, \delta, q, S, \emptyset)$$

δ is defined by the following rules:

$$R_1: \delta(q, \Lambda, S) = \{(q, 0BB)\}$$

$$R_2: \delta(q, \Lambda, B) = \{(q, 0S), (q, 1S), (q, 0)\}$$

$$R_3: \delta(q, 0, 0) = \{(q, \Lambda)\}$$

$$R_4: \delta(q, 1, 1) = \{(q, \Lambda)\}$$

$$\begin{aligned} (q, 010^4, S) &\rightarrow (q, 010^4, 0BB) && \text{by rule } R_1 \\ &\rightarrow (q, 10^4, BB) && \text{by rule } R_3 \\ &\rightarrow (q, 10^4, 1SB) && \text{by rule } R_2 \\ &\rightarrow (q, 0^4, SB) && \text{by rule } R_4 \\ &\rightarrow (q, 0^4, 0BBB) && \text{by rule } R_1 \\ &\rightarrow (q, 0^3, BBB) && \text{by rule } R_3 \\ &\rightarrow (q, 0^3, 0BB) && \text{by rule } R_2 \text{ since } (q, 0) \in \delta(q, \Lambda, B) \end{aligned}$$

$\rightarrow (q, 0^2, BB)$ by rule R_3

$\rightarrow (q, 0^2, 0B)$ by rule R_2

$\rightarrow (q, 0, B)$ by rule R_2

$\rightarrow (q, 0, 0)$ by rule R_2

$\rightarrow (q, \Lambda, \Lambda)$

Thus $010^4 \in N(A)$

COPPERBELT UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

COMPILER CONSTRUCTION AND THEORY OF AUTOMATA (CS 314)

LECTURE NOTES

UNIT 10: TURING MACHINES

INTRODUCTION

So far in our development of the theory of computation we have presented several models of computing devices. Finite automata are good models for devices that have a small amount of memory. Pushdown automata are good models for devices that have an unlimited memory that is usable only in the last in, first out manner of a stack. However, these models are too restricted to serve as models of general purpose computers.

We will now consider a much more powerful model, first proposed by Alan Turing in 1936, called the Turing machine. Similar to a finite automaton but with an unlimited and unrestricted memory, a Turing machine is a much more accurate model of a general purpose computer. A Turing machine can do everything that a real computer can do.

The Turing machine model uses an infinite tape as its unlimited memory. It has a tape head that can read and write symbols and move around on the tape. Initially the tape contains only the input string and is blank everywhere else. If the machine needs to store information, it may write this information on the tape. To read the information that it has written, the machine can move its head back over it. The machine continues computing until it decides to produce an output. The outputs *accept* and *reject* are obtained by entering designated accepting and rejecting states. If it doesn't enter an accepting or a rejecting state, it will go on forever, never halting. Figure 23 illustrates a Turing machine model.

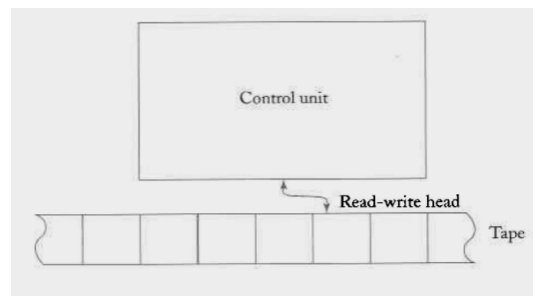


Figure 23 Turing Machine Model

The following list summarizes the differences between finite automata and Turing machines.

1. A Turing machine can both write on the tape and read from it.
2. The read-write head can move both to the left and to the right.
3. The tape is infinite.
4. The special states for rejecting and accepting take effect immediately.

FORMAL DEFINITION OF A TURING MACHINE

A Turing machine M is a 7-tuple, viz. $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$ where

Q is a finite nonempty set of states,

Σ is the nonempty set of input alphabet,

Γ is a finite nonempty set of tape symbols,

δ is the transition function mapping the states of the finite automaton and tape symbols to states, tape symbols and movement of the head, i.e. $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$,

$b \in \Gamma$ is the blank symbol,

$q_0 \in Q$ is the initial state, and

$F \subseteq Q$ is the set of final states.

REPRESENTATION OF TURING MACHINES

We can describe a Turing machine using (i) instantaneous descriptions using move-relations, (ii) transition table, and (iii) transition diagram.

REPRESENTATION BY INSTANTANEOUS DESCRIPTION

‘Snapshots’ of a Turing machine in action can be used to describe a Turing machine. These give “instantaneous description” of a Turing machine. An instantaneous description of a Turing machine is defined in terms of the entire input string and the current state.

Definition: an instantaneous description of a Turing machine M is a string $\alpha\beta\gamma$, where β is the present state of M , the entire input string is split as $\alpha\gamma$, the first symbol of γ is the current symbol under R/W head and γ has all the subsequent symbols of the input string, and the string α is the substring of the input string formed by all the symbols to the left of α .

Example: A snapshot of Turing machine is shown in figure 24. Obtain an instantaneous description.

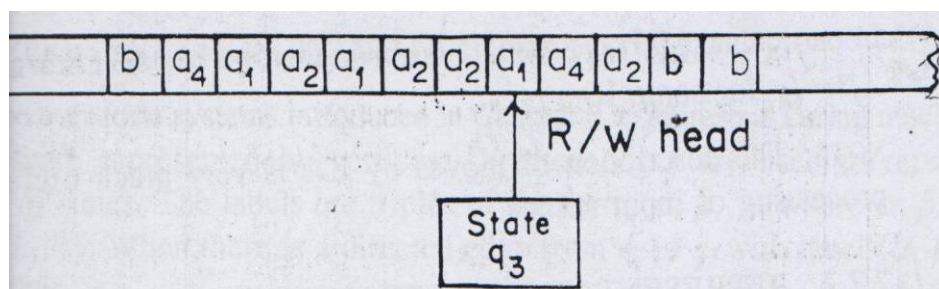


Figure 24: A snapshot of Turing Machine

Solution: The present symbol under the R/W head is a_1 . The present state is q_3 . So a_1 is written to the right of q_3 . The nonblank symbols to the left of a_1 form the string $a_4a_3a_2$, which is written to the left of q_3 . The sequence of nonblank symbols to the right of a_1 is a_4a_2 . Thus the ID is given in figure 25.

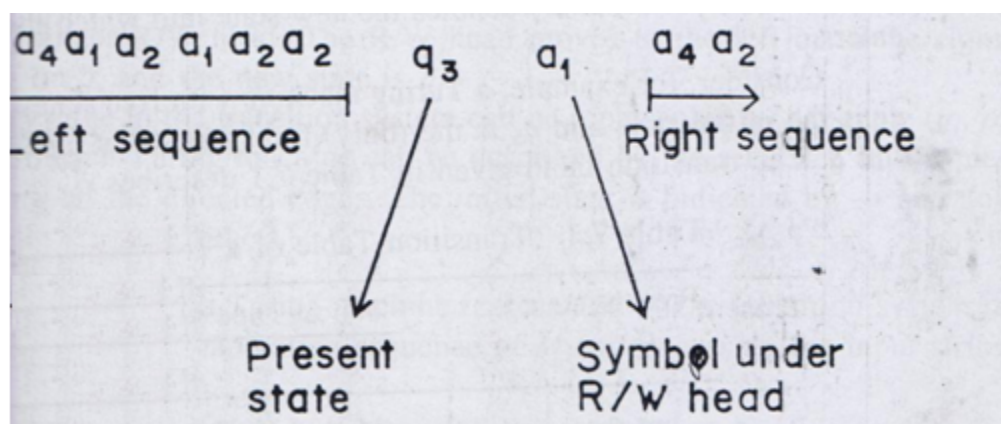


Figure 25 Representation of ID

Moves in a TM

As in the case of Pushdown Automata, $\delta(q, x)$ induces a change in ID of the Turing machine. We call this change in ID a move. the next move function specifies the change in the ID. The next move function says that, when the machine is in state q and scans symbol a on a cell, then the machine writes a symbol b instead of a , moves one cell in the direction left or right, and enters state p .

REPRESENTATION BY TRANSITION SYSTEM

We give the definition of δ in the form of a table called the transition table. If $\delta(q, a) = (\gamma, \alpha, \beta)$, we write $\alpha\beta\gamma$ under a -column and q -row. So if we get $\alpha\beta\gamma$ in the table, it means that α is written in the current cell, β gives the movement of the head (L or R) and γ denotes the new state into which the Turing machine enters.

Consider, for example, a Turing machine with five states $q_1 \dots q_5$, where q_1 is the initial state and q_5 is the final state. The tape symbols are 0,1 and b. The transition table given in Table 6 describes δ .

Present State	Tape Symbols		
	B	0	1
$\rightarrow q_1$	1L q_2	0R q_1	
q_2	bR q_3	0L q_2	1L q_2
q_3		bR q_4	bR q_5
q_4	0R q_5	0R q_4	1R q_4
	0L q_2		

Table 6 Transition Table of a Turing Machine

Example: Consider the TM description given in table 6. Draw the computation sequence of the input string 00b.

$q_1 00b \vdash 0q_1 0b \vdash 00q_1 b \vdash 0q_2 01 \vdash q_2 001$
 $\vdash q_2 b 001 \vdash bq_3 001 \vdash bbq_4 01 \vdash bb0q_4 1 \vdash bb01q_4 b$
 $\vdash bb010q_5 b \vdash bb01q_2 00 \vdash bb0q_2 100 \vdash bbq_2 0100$
 $\vdash bq_2 b 0100 \vdash bbq_3 0100 \vdash bbbq_4 100 \vdash bbb1q_4 00$
 $\vdash bbb10q_4 0 \vdash bbb100q_4 b \vdash bbb1q_2 00$
 $\vdash bbb100q_2 00 \vdash bbb10q_2 000 \vdash bbb1q_2 0000$
 $\vdash bbbq_2 10000 \vdash bbq_2 b 10000 \vdash bbbq_3 10000 \vdash bbbq_5 0000.$

REPRESENTATION BY TRANSITION DIAGRAM

We can use transition systems to represent Turing machines. The states are represented by vertices. The directed edges are used to represent transition of states. The labels are triples of the form (α, β, γ) , where $\alpha, \beta \in \Gamma$ and $\gamma \in \{L, R\}$. When there is a directed edge from q_i to q_j with label (α, β, γ) , it means that

$$\delta(q_i, \alpha) = (q_j, \beta, \gamma)$$

During the processing of an input string, suppose the Turing machine enters q_i and R/W head scans the (present) symbol α . As a result, the symbol β is written in the cell under the R/W head. The R/W head moves to the left or to the right, depending on γ , and the new state is q_j .

Example: M is a Turing machine represented by the transition system in figure 26. Obtain the computation sequence of M for processing the input string 0011.

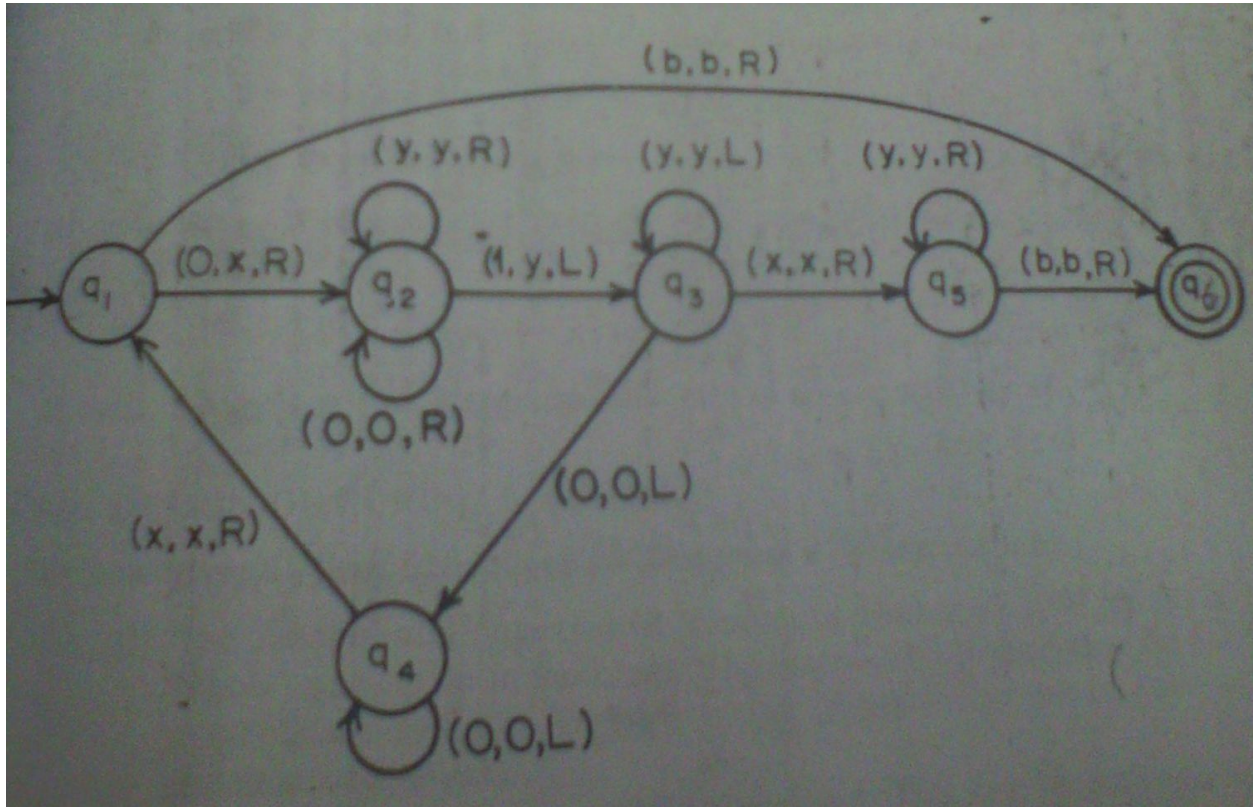


Figure 26 Transition system for M

The computation sequence is given figure 27.

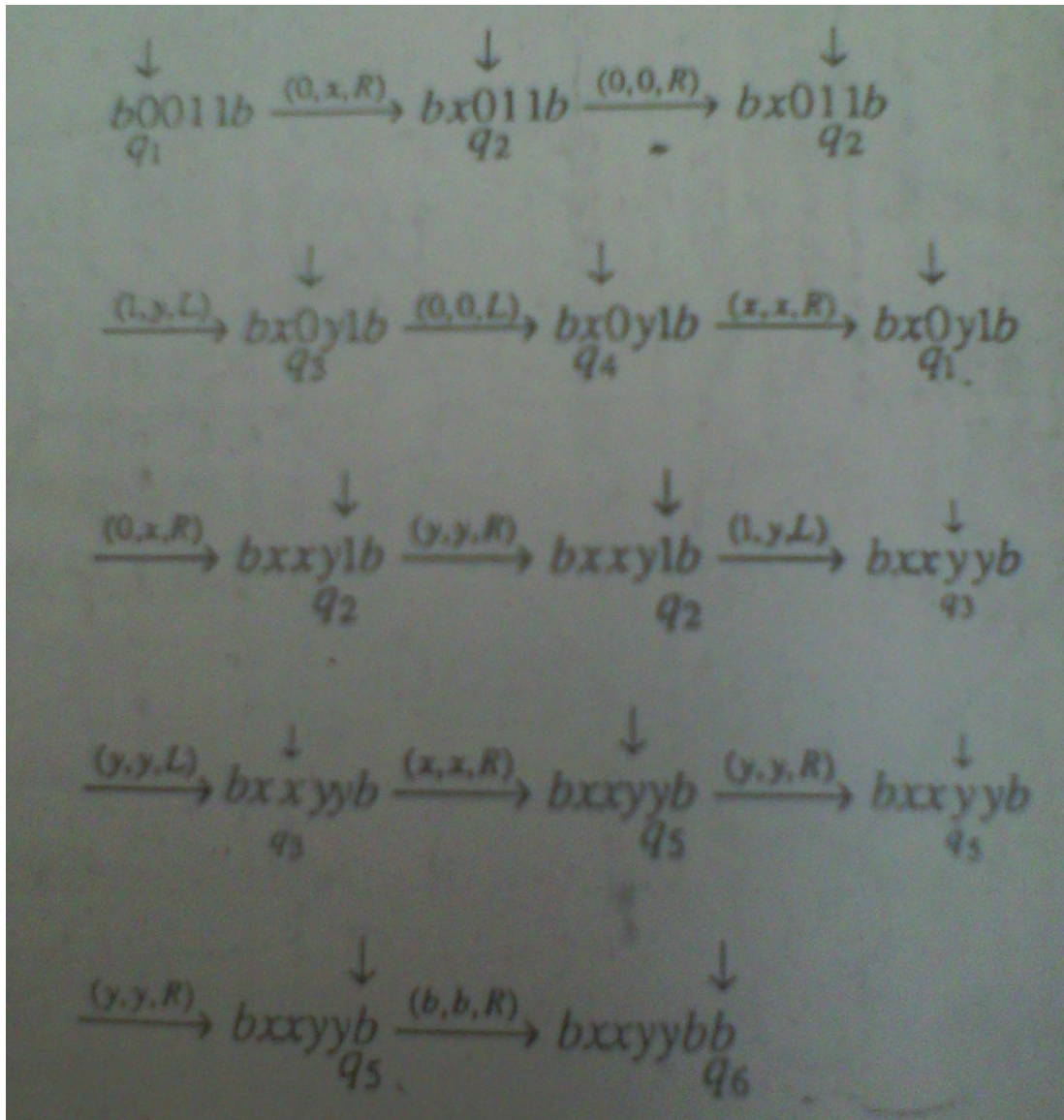


Figure 27

LANGUAGE ACCEPTABILITY BY TURING MACHINE

Lets us consider the Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$. A string w in Σ^* is said to be accepted by M if $q_0 w \xrightarrow{*} \alpha_1 p \alpha_2$ for some $p \in F$ and $\alpha_1, \alpha_2 \in \Gamma^*$. M does not accept w if the machine either halts in a nonaccepting state or does not halt.

HALTING PROBLEM OF TURING MACHINES

We have learned that deterministic Turing machines are capable of doing any computation that computers can do, that is computationally they are equally powerful, and that any of their variations do not exceed the computational power of deterministic Turing machines. there are problems that cannot be solved by Turing machines hence by computers. Here "unsolvability" is

in the following sense. First recall that solving a problem can be viewed as recognizing a language. So we are going to look at the unsolvability in terms of language recognition. Suppose that a language is acceptable but not decidable. Then given a string a Turing machine that accept the language starts the computation. At any point in time, if the Turing machine is running, there is no way of telling whether it is in an infinite loop or along the way to a solution and it needs more time. Thus if a language is not decidable, the question of whether or not a string is in the language may not be answered in any finite amount of time. Since we can not wait forever for an answer, the question is unanswerable that is the problem is unsolvable. Below we are going to see some well known unsolvable problems and see why we can say they are unsolvable.

Halting Problem

One of well known unsolvable problems is the halting problem. It asks the following question: Given an arbitrary Turing machine M over alphabet $\Sigma = \{a, b\}$, and an arbitrary string w over Σ , does M halt when it is given w as an input ?

Definition: A class of problems with two outputs (yes/no) is said to be solvable (decidable) if there exists some definite algorithm which always terminates (halts) with two outputs (yes/no). Otherwise, the class of problems is said to be unsolvable (undecidable).

Turing Theorem: The halting problem of Turing machine over $\Sigma = \{0,1\}$ is unsolvable (i.e. the problem of determining whether or not an arbitrary Turing machine M over $\{0,1\}$ halts for an arbitrary input x in Σ^* is unsolvable).